

大模型学习笔记

李东泽 241240041@smail.nju.edu.cn

2026 年 3 月 17 日

目录

1	Introduction	6
1.1	Why This Course Exists	6
1.2	Industrialization of Language Models	6
1.3	What Transfers to Frontier Models	6
1.4	Efficiency as the Central Principle	7
1.5	Historical Landscape of Language Models	7
1.6	Levels of Openness	7
1.7	Executable Lecture and Course Structure	8
1.8	Summary	8
2	Tokenization	9
2.1	Why Tokenization Is Needed	9
2.2	What Makes a Good Tokenizer	9
2.3	Examples and Intuition	10
2.4	Character-Based Tokenization	10
2.5	Byte-Based Tokenization	11
2.6	Word-Based Tokenization	11
2.7	Byte Pair Encoding (BPE): The Main Practical Compromise	12
2.8	Training a BPE Tokenizer	12
2.9	The Merge Operation	13
2.10	Encoding and Decoding with BPE	13
2.11	Pre-tokenization and GPT-2 Style Segmentation	14
2.12	Implementation Advice and Engineering Suggestions	14
2.13	Conceptual Summary	15
2.14	Summary	15
3	Lecture 02, Part I: Motivation, Tensor Basics, and Memory Accounting	16
3.1	Overview of This Part	16
3.2	Motivating Questions: Why Resource Accounting Matters	16
3.2.1	Question 1: Training a 70B model on 15T tokens with 1024 H100s	16

3.2.2	Question 2: How large a model can 8 H100s hold with AdamW (naively)?	17
3.3	Tensor Basics	18
3.3.1	How tensors are created	19
3.4	Tensor Memory Accounting	19
3.4.1	float32	20
3.4.2	float16	21
3.4.3	bfloat16	21
3.4.4	fp8	22
3.5	What This Part Really Wants Us to Learn	22
3.5.1	(1) Always do rough resource accounting first	22
3.5.2	(2) Tensor is the universal storage unit	22
3.5.3	(3) Memory accounting is simple in form, but powerful in effect	22
3.5.4	(4) Precision choice is an engineering tradeoff	23
3.6	Summary Notes	23
4	Lecture 02, Part II: Tensors on GPUs and Basic Tensor Operations	24
4.1	Overview of This Part	24
4.2	Tensors on GPUs	24
4.2.1	Default device: CPU	24
4.2.2	Why move tensors to GPU?	25
4.2.3	Moving tensors to GPU	26
4.2.4	Memory accounting when moving to GPU	26
4.3	Tensor Operations	27
4.4	Tensor Storage	27
4.4.1	What is a tensor in PyTorch?	27
4.4.2	Stride	28
4.5	Tensor Slicing and Views	28
4.5.1	Views do not copy memory	29
4.5.2	Non-contiguous tensors	30
4.5.3	contiguous() creates a copy	30
4.6	Elementwise Operations	31
4.7	Matrix Multiplication	32
4.7.1	Basic example	32
4.7.2	Batched matrix multiplication	32
4.8	What This Part Really Wants Us to Learn	33
4.8.1	(1) Device matters	33
4.8.2	(2) Tensor is storage plus metadata	33
4.8.3	(3) View is cheap, copy is expensive	33
4.8.4	(4) Matrix multiplication dominates	34
4.9	Summary Notes	34

5	Lecture 02, Part III: Tensor Dimensions, Jaxtyping, and Einops	35
5.1	Overview of This Part	35
5.2	Motivation: Why Ordinary PyTorch Code Becomes Hard to Read	35
5.3	Jaxtyping Basics	36
5.3.1	Old way vs new way	36
5.4	Einops	37
5.5	Einops Einsum	38
5.5.1	The setup	38
5.5.2	Old way	38
5.5.3	New (einops) way	38
5.5.4	Using ...	39
5.6	Einops Reduce	39
5.6.1	The setup	39
5.6.2	Old way	39
5.6.3	New (einops) way	40
5.7	Einops Rearrange	40
5.7.1	Motivation	40
5.7.2	Step 1: Split a dimension	41
5.7.3	Step 2: Apply a transformation on one sub-dimension	41
5.7.4	Step 3: Merge dimensions back	42
5.8	What This Part Really Wants Us to Learn	42
5.8.1	(1) Dimension bookkeeping is a real problem	43
5.8.2	(2) Named dimensions improve readability	43
5.8.3	(3) Einsum expresses contraction clearly	43
5.8.4	(4) Rearrange expresses shape transformation semantically	43
5.8.5	(5) Documentation matters even without enforcement	43
5.9	Summary Notes	43
6	Lecture 02, Part IV: Compute Accounting, FLOPs, and Gradients	45
6.1	Overview of This Part	45
6.2	What is a FLOP?	45
6.3	Large-Scale Intuition: Why FLOPs Matter	46
6.4	A First-Principles Example: Linear Model FLOPs	46
6.4.1	The setup	47
6.4.2	Counting FLOPs	47
6.4.3	Other operations	48
6.4.4	Interpretation	48
6.5	From FLOPs to Wall-Clock Time	49
6.5.1	Promised FLOP/s	49
6.5.2	MFU: Model FLOPs Utilization	50
6.5.3	bfloat16 comparison	50

6.6	Summary of Tensor Operation FLOPs	51
6.7	Gradients Basics	51
6.7.1	A scalar example	51
6.8	Gradients FLOPs	52
6.8.1	The model	53
6.8.2	Forward FLOPs	53
6.8.3	Backward for w_2	53
6.8.4	Backward for h_1	54
6.8.5	Backward for w_1	54
6.8.6	Putting it together	54
6.9	What This Part Really Wants Us to Learn	55
6.9.1	(1) FLOPs can be counted systematically	55
6.9.2	(2) Matmul is the dominant compute term	55
6.9.3	(3) Hardware throughput depends on both device and dtype	55
6.9.4	(4) Backward is expensive	55
6.9.5	(5) The famous $6ND$ rule has a real derivation	56
6.10	Summary Notes	56
7	Lecture 02, Part V: Parameters, Initialization, and Building a Simple Model	57
7.1	Overview of This Part	57
7.2	Model Parameters in PyTorch	57
7.3	Parameter Initialization	58
7.3.1	Naive initialization and its problem	58
7.3.2	Rescaling by $1/\sqrt{input_dim}$	59
7.3.3	Relation to Xavier initialization	60
7.3.4	Truncated normal for extra safety	60
7.4	Building a Custom Model	61
7.4.1	A simple custom linear layer	61
7.4.2	A simple deep linear model: Cruncher	61
7.4.3	Why <code>ModuleList</code> ?	63
7.4.4	The final <code>squeeze</code>	63
7.5	Inspecting Model Parameters	63
7.5.1	Parameter sizes via <code>state_dict</code>	64
7.5.2	Total number of parameters	64
7.6	Moving the Model to GPU	65
7.7	Running the Model	65
7.8	What This Part Really Wants Us to Learn	66
7.8.1	(1) Parameters are special tensors	66
7.8.2	(2) Initialization is about signal scale	66
7.8.3	(3) Models are structured parameter collections	66
7.8.4	(4) Module registration matters	66

7.8.5	(5) Shape discipline matters	66
7.9	Summary Notes	67

1 Introduction

本讲在进入具体技术内容（尤其是 tokenization）之前，首先回答一个更根本的问题：为什么今天仍然需要系统地学习如何从零构建语言模型？这部分内容并不直接涉及某一个模块的实现细节，而是为整门课程建立总体背景、历史脉络与方法论框架。

1.1 Why This Course Exists

课程首先指出，当代研究者与语言模型底层技术之间正在逐渐脱节。若干年前，研究者通常需要亲自实现模型并完成训练；随后，主流工作方式变成下载预训练模型并在特定任务上进行微调；而到了今天，越来越多的人只需要通过 prompt 使用闭源的大模型系统即可完成任务。这种抽象层次的提升固然显著提高了生产力，但也带来了一个直接后果：研究者可能越来越难以把握模型训练、推理、系统优化与数据处理等底层机制。

然而，与成熟的软件抽象不同，语言模型领域的高层接口并不是完全稳定且封闭良好的。许多核心问题仍然开放，例如架构设计、训练稳定性、并行化策略、推理优化、数据配比以及对齐方法等。因此，若想进行更本质的研究，仅仅会“调用模型”是不够的，还必须理解模型为何如此工作、其性能瓶颈在哪里、以及不同设计决策背后的约束条件。课程因此提出一个核心理念：

理解来自构建（understanding via building）。

也就是说，真正的理解并非停留在阅读论文或使用 API 的层面，而是来自于亲手实现与训练一个完整系统。

1.2 Industrialization of Language Models

课程接着强调，语言模型已经进入高度工业化阶段。最前沿的大模型往往拥有极大的参数规模、极高的训练成本，以及只有少数公司才能承担的硬件与基础设施投入。这意味着，对于普通学生或研究者而言，完整复现 frontier model 几乎是不现实的。同时，许多最先进系统的训练细节、数据来源和工程细节并未公开，因此外界往往只能看到结果，而难以看到其实现路径。

这引出了一个重要问题：既然无法直接构建 frontier model，那么在课堂上训练一个小得多的模型是否仍然有意义？课程给出的回答是：有意义，但必须明确其边界。小模型并不总能完全代表大模型，因为不同规模下的计算分布、系统瓶颈，甚至模型能力本身都可能发生变化。例如，随着模型规模增加，attention 与 MLP 的计算占比会变化；某些能力可能只在更大尺度下表现出明显的“涌现”现象。因此，课堂中的小规模实验并不是对大模型的简单缩放，而是一种用于建立基本机制理解与研究思维的载体。

1.3 What Transfers to Frontier Models

课程将可以学习的内容分为三类：

1. **Mechanics**: 机制层面的知识，即模型与系统究竟如何工作；
2. **Mindset**: 研究与工程思维，即面对大规模系统时应如何分析资源、权衡设计并优化性能；

3. Intuitions: 经验性的直觉，即哪些设计在实践中往往更有效。

其中，mechanics 与 mindset 具有较强的可迁移性，因此是课程最核心的教学目标；而 intuitions 则部分依赖具体规模与实验环境，不一定能够从小模型直接外推到大模型。课程对此保持谨慎态度：它希望训练学生理解系统原理并建立正确的研究视角，而不是简单灌输某些可能随规模变化而失效的经验法则。

1.4 Efficiency as the Central Principle

整门课程的主线可以用一句话概括：**一切都围绕效率展开**。课程给出一个非常重要的观点：模型效果并不仅仅取决于投入多少资源，更取决于我们是否能够高效地利用这些资源。资源包括数据、计算量、显存、通信带宽以及训练时间等。在资源预算固定的情况下，真正决定模型上限的，往往是算法与系统设计的效率。

这意味着，课程不会把“大模型更强”简单理解为“参数更多就行”，而是始终关注这样一个问题：

在给定数据与硬件预算的前提下，如何构建出效果最好的模型？

因此，无论是后续的 tokenization、模型架构、训练策略、并行化方案、推理优化，还是数据清洗与对齐方法，本质上都将被放在“效率最大化”这一统一框架下理解。

1.5 Historical Landscape of Language Models

为了帮助学生建立全景视角，课程还回顾了语言模型的发展历史。首先是在神经网络广泛应用之前，研究主要围绕信息论与 n-gram 统计语言模型展开；随后进入神经网络时代，出现了早期 neural language model、sequence-to-sequence、attention、Adam 优化器以及 Transformer 等关键技术，这些构成了现代语言模型的核心基础设施。再往后，预训练与迁移学习逐渐成为主流，ELMo、BERT、T5 等模型推动了 foundation model 范式的发展。

在此基础上，GPT-2、GPT-3、PaLM、Chinchilla 等模型进一步展示了规模扩展的重要性，并促使研究者系统研究 scaling laws。与此同时，开源生态也逐渐发展起来，例如 The Pile、GPT-J、OPT、BLOOM、Llama、Qwen、DeepSeek 和 OLMo 等项目，分别从数据、权重、训练细节与开放程度等方面推动了开放研究。通过这条历史线索，课程想强调的是：今天的大模型并不是某个单一想法的结果，而是长期积累的架构创新、训练方法、系统工程和数据工程共同推动的产物。

1.6 Levels of Openness

课程还区分了不同层次的“开放性”。闭源模型通常只提供 API；开放权重模型会提供模型参数以及部分架构说明，但可能不公开训练数据与全部训练细节；真正意义上的开源模型则进一步公开数据、代码与更多实验细节。这一区分对研究非常重要，因为可解释、可复现、可修改的程度，直接影响学生和 researcher 能够在多大程度上理解并推进该领域的发展。

1.7 Executable Lecture and Course Structure

最后，这一部分还介绍了课程本身的组织形式。CS336 的讲义并不只是静态幻灯片，而是“可执行讲义”（executable lecture）：课程内容本身通过程序结构组织起来，运行代码即可得到讲义输出。这种形式强调，概念、实现与实验是统一的，学生在学习过程中不仅阅读内容，而且可以直接进入代码、观察变量、运行示例并追踪讲义结构。

在课程整体结构上，后续内容将围绕五个核心模块展开：

1. **Basics**: 从零实现基本的语言模型训练流水线；
2. **Systems**: 研究 kernel、并行化与推理优化；
3. **Scaling Laws**: 通过小规模实验预测更大规模下的最优配置；
4. **Data**: 研究数据来源、处理、过滤、去重与评测；
5. **Alignment**: 研究如何让 base model 更加有用、可控且安全。

这五个模块共同构成了“从零构建语言模型”的完整路径，而 tokenization 则是进入该路径后的第一个具体技术主题。

1.8 Summary

综上，tokenization 之前的内容承担的是“搭框架”的作用。它并不急于进入某一个局部技术细节，而是先让学生明白：为什么要学这门课、当前语言模型领域处于怎样的历史与工业背景中、哪些知识真正具有迁移价值，以及为什么课程会把“效率”视为贯穿始终的核心原则。在这个框架建立之后，课程才自然过渡到第一个具体问题：如何把原始文本表示成模型能够处理的 token 序列。

2 Tokenization

在完成课程的总体背景介绍之后，讲义进入第一个具体技术主题：**tokenization**。这一部分的任务，是解释为什么语言模型不能直接处理原始字符串、tokenizer 在整个语言模型系统中扮演什么角色，以及现代语言模型为何通常采用 Byte Pair Encoding (BPE) 作为实际可行的折中方案。与前面的 introduction 相比，这一章开始真正进入“从零构建语言模型”的技术细节，因此不仅需要理解概念，还需要关注其实现方式与工程约束。

2.1 Why Tokenization Is Needed

原始文本在程序中通常表示为 Unicode 字符串。然而，语言模型本质上并不是在字符串空间上直接建模，而是在一个离散符号序列上建模。更准确地说，模型所学习的是一个 token 序列的概率分布，例如

$$P(t_1, t_2, \dots, t_n),$$

其中每个 t_i 都是一个离散 token 的索引，而不是原始字符本身。

因此，在字符串与模型输入之间，必须存在一个中间层，它完成两个方向的映射：

- **encode**: 将字符串映射为 token 序列；
- **decode**: 将 token 序列还原为字符串。

讲义中首先将这种接口抽象为一个统一的 `Tokenizer` 类：

```
class Tokenizer(ABC):
    """Abstract interface for a tokenizer."""
    def encode(self, string: str) -> list[int]:
        raise NotImplementedError
    def decode(self, indices: list[int]) -> str:
        raise NotImplementedError
```

这个抽象非常重要，因为它表明：无论底层采用字符、字节、单词还是子词，tokenizer 对外都应当表现为同一个接口。后续无论更换何种 tokenization 策略，只要满足 encode/decode 的约定，模型的其余部分就能够保持相对稳定。

2.2 What Makes a Good Tokenizer

一个好的 tokenizer 通常需要同时满足几个彼此制约的目标：

1. **词表不能太大**：若 vocabulary 太大，embedding 层和输出层会非常昂贵；
2. **序列不能太长**：若 token 数过多，则 Transformer 的注意力计算和显存开销都会迅速增加；
3. **必须可逆**：encode 后的 token 序列应当能够通过 decode 准确恢复原始字符串；
4. **应对高频模式有压缩能力**：常见的字符组合、前后缀、单词片段最好能被较少 token 表示。

这四点决定了 tokenization 本质上不是一个纯粹的字符串处理问题，而是一个同时受模型架构、资源预算与训练效率约束的设计问题。

讲义用一个简单的“压缩率”概念来帮助理解 tokenization 的优劣：

```
def get_compression_ratio(string: str, indices: list[int]) -> float:
    num_bytes = len(bytes(string, encoding="utf-8"))
    num_tokens = len(indices)
    return num_bytes / num_tokens
```

若该比值较大，说明平均每个 token 可以承载更多字节信息，序列更短；若该比值接近 1，则意味着 tokenization 几乎没有压缩效果。对基于 Transformer 的模型来说，更高的压缩率往往意味着更低的上下文长度成本，因此这一指标具有直接的工程意义。

2.3 Examples and Intuition

讲义先通过 GPT-2 tokenizer 的例子帮助学生建立直觉。其核心观察包括：

- 一个单词与其前导空格经常共同构成 token；
- 处于句首和句中的同一个单词可能会被编码为不同 token；
- 数字通常按几位一组切分，而非完全逐字符切分；
- tokenizer 并不是按“人类语言学中的单词边界”机械切分，而是按统计规律组织 token。

这些现象提醒我们：现代 tokenizer 的目标不是追求语言学上最自然的切分，而是追求统计有效性与计算效率。

2.4 Character-Based Tokenization

讲义首先介绍字符级 tokenization。其思想是：把 Unicode 字符串视为 Unicode code point 序列，每个字符对应一个整数。代码实现非常直接：

```
class CharacterTokenizer(Tokenizer):
    def encode(self, string: str) -> list[int]:
        return list(map(ord, string))
    def decode(self, indices: list[int]) -> str:
        return "".join(map(chr, indices))
```

字符级 tokenization 的优点是概念简单、可逆、不会出现“未知字符无法编码”的问题。但它也存在明显缺陷。首先，Unicode 字符集合很大，导致词表规模偏大；其次，很多字符极为稀有，使得词表利用率低；最后，字符粒度过细，意味着序列往往较长，模型需要跨越大量 token 才能表达一个常见词或语义单元。因此，字符级方法虽然易于理解，但通常不是现代大模型的首选方案。

2.5 Byte-Based Tokenization

为了解决字符集过大的问题，讲义接着介绍字节级 tokenization。由于任意 Unicode 字符串都可以用 UTF-8 表示，因此我们可以直接把字符串转化为字节序列，再将每个字节映射为 0 到 255 之间的整数。其实现如下：

```
class ByteTokenizer(Tokenizer):
    def encode(self, string: str) -> list[int]:
        string_bytes = string.encode("utf-8")
        indices = list(map(int, string_bytes))
        return indices

    def decode(self, indices: list[int]) -> str:
        string_bytes = bytes(indices)
        string = string_bytes.decode("utf-8")
        return string
```

字节级 tokenization 的主要优点是词表规模固定为 256，极其简洁，而且天然支持所有 UTF-8 文本，不存在 OOV (out-of-vocabulary) 问题。但它的主要缺点同样非常突出：**压缩率极差**。讲义中甚至直接验证了这一点：字节级 tokenization 的 compression ratio 恰好为 1。这意味着每个 token 仅对应一个字节，序列长度不会得到任何缩短。由于 Transformer 的计算与 token 数密切相关，字节级方法在当前主流架构下往往过于低效。

2.6 Word-Based Tokenization

随后，讲义介绍了更接近传统 NLP 的单词级 tokenization。直观上，我们似乎可以先通过正则表达式把文本切成单词与标点，再将这些片段映射为整数。例如：

```
segments = regex.findall(r"\w+|.", string)
```

讲义还展示了 GPT-2 风格的更复杂预切分正则：

```
GPT2_TOKENIZER_REGEX = \
    r"''('?:[sdmt]|ll|ve|re)| ?\p{L}+| ?\p{N}+| ?[^\s\p{L}\p{N}]+\s+(?!\\S)|\\s+''''"
```

单词级方法的优点是序列长度相对较短，也更符合初学者的直觉。然而，它存在三个根本问题：

1. 词表大小会非常大，因为自然语言中的单词种类极多；
2. 长尾问题严重，大量低频词在训练中难以学好；
3. 未登录词 (OOV) 不可避免，往往需要引入 UNK token，从而损失信息。

因此，纯单词级方法在现代大模型中也不是理想选择。

2.7 Byte Pair Encoding (BPE): The Main Practical Compromise

在比较了字符级、字节级和单词级之后，讲义引出真正重要的方案：**Byte Pair Encoding (BPE)**。BPE 的基本思想是，从较小的基本单位（本讲义中从字节开始）出发，反复把语料中最常见的相邻 token 对合并为一个新 token。这样，常见模式会被压缩成更大的 token，而低频模式仍然可以退回到较小单位表示。

因此，BPE 试图在下列矛盾之间取得平衡：

- 避免字符级与字节级那样序列过长；
- 避免单词级那样词表过大与 OOV 问题严重；
- 在统计意义上，让高频模式拥有更紧凑的表示。

从工程角度看，这正是现代 tokenizer 成功的关键：它不是追求语言学上“最自然”的分词方式，而是追求在固定词表预算下最大化表达效率。

2.8 Training a BPE Tokenizer

讲义通过一个小例子解释了 BPE 的训练过程。其核心函数如下：

```
def train_bpe(string: str, num_merges: int) -> BPETokenizerParams:
    indices = list(map(int, string.encode("utf-8")))
    merges: dict[tuple[int, int], int] = {}
    vocab: dict[int, bytes] = {x: bytes([x]) for x in range(256)}
    for i in range(num_merges):
        counts = defaultdict(int)
        for index1, index2 in zip(indices, indices[1:]):
            counts[(index1, index2)] += 1
        pair = max(counts, key=counts.get)
        index1, index2 = pair
        new_index = 256 + i
        merges[pair] = new_index
        vocab[new_index] = vocab[index1] + vocab[index2]
        indices = merge(indices, pair, new_index)
    return BPETokenizerParams(vocab=vocab, merges=merges)
```

这一过程可以拆解为四步：

1. 从字节序列开始，把每个字节看作初始 token；
2. 统计当前序列中每个相邻 token 对出现的次数；
3. 找到出现次数最多的 token 对，并为其分配一个新的 token id；
4. 用这个新 token 替换序列中所有该相邻对的出现位置。

重复进行若干轮之后，就得到一套 merges 规则与扩展后的 vocabulary。这里的训练过程本质上是一种基于语料统计的压缩启发式：高频相邻模式被逐步凝聚成新的基本单元。

2.9 The Merge Operation

理解 BPE 的关键，在于理解“合并”本身如何发生。讲义中的 `merge` 函数如下：

```
def merge(indices: list[int], pair: tuple[int, int], new_index: int) -> list[int]:
    new_indices = []
    i = 0
    while i < len(indices):
        if i + 1 < len(indices) and indices[i] == pair[0] and indices[i + 1] == pair[1]:
            new_indices.append(new_index)
            i += 2
        else:
            new_indices.append(indices[i])
            i += 1
    return new_indices
```

这个实现虽然简单，却非常具有教学价值。它做的事情是：顺序扫描 token 序列，一旦发现当前位置与下一位置恰好构成待合并的 `pair`，就把这两个 token 替换为一个新的 token；否则就原样保留当前 token。通过这一过程，序列会逐步缩短，而更大的 token 单元会逐渐形成。

从原理上看，BPE 的训练就是不断地定义新的“宏符号”。原先需要两个位置表示的高频结构，现在可以由一个新的 token 表示，因此序列压缩率会提高。这也是 BPE 相比纯字节方案更有效的根本原因。

2.10 Encoding and Decoding with BPE

训练完 BPE 之后，tokenizer 的 `encode` 过程就是：先把字符串转成字节序列，再按照训练时记录的 `merges` 顺序反复应用合并规则。讲义中的代码为：

```
class BPETokenizer(Tokenizer):
    def __init__(self, params: BPETokenizerParams):
        self.params = params

    def encode(self, string: str) -> list[int]:
        indices = list(map(int, string.encode("utf-8")))
        for pair, new_index in self.params.merges.items():
            indices = merge(indices, pair, new_index)
        return indices

    def decode(self, indices: list[int]) -> str:
        bytes_list = list(map(self.params.vocab.get, indices))
        string = b"".join(bytes_list).decode("utf-8")
        return string
```

这里需要特别注意两个点。第一，`encode` 的实现虽然正确，但很慢，因为它对每个 `merge` 都重新扫描整个序列；第二，`decode` 的正确性依赖于 `vocab` 中保存了每个 `token` 对应的完整字节串。也就是说，BPE 的 `decode` 并不需要“反向拆分树”，只要知道每个 `token` 最终表示的字节串即可。

2.11 Pre-tokenization and GPT-2 Style Segmentation

课程中还提到，实际工业级 `tokenizer` 通常不会直接在整段原始字节流上做 BPE，而会在 BPE 之前引入预切分（pre-tokenization）。例如 GPT-2 的 `tokenizer` 通过复杂正则表达式，先把文本切成适当的小段，再在段内做 BPE。这种做法有两个作用：

- 控制 `merge` 的统计范围，使其更符合常见文本模式；
- 更好地保留空格、数字和标点等结构信息。

这说明实际 `tokenizer` 通常不是一个单一算法，而是若干层处理的组合：预切分、字节化、子词合并、特殊 `token` 处理等。

2.12 Implementation Advice and Engineering Suggestions

讲义最后特别强调：当前代码是一个**教学实现**，其首要目标是帮助学生理解原理，而不是追求工业级性能。因此，如果你要在 `assignment` 或自己的项目中实现 `tokenizer`，可以从以下几个方向逐步改进：

(1) 先保证正确性，再考虑速度。最重要的是确保 `encode` 与 `decode` 能严格 round-trip，即

$$\text{decode}(\text{encode}(\text{string})) = \text{string}.$$

在此基础上，再考虑性能优化。否则，即使 `tokenizer` 很快，只要不能稳定恢复原始文本，就无法用于真实训练流程。

(2) 明确区分训练阶段与使用阶段。`train_bpe` 的任务是从语料中学习 `merges` 和 `vocabulary`；`BPETokenizer` 的任务则是使用已有参数进行 `encode/decode`。实现时一定要保持这种分层清晰，否则后续维护和调试会变得困难。

(3) 不要在 `encode` 时无脑遍历所有 `merges`。当前教学代码中：

```
for pair, new_index in self.params.merges.items():
    indices = merge(indices, pair, new_index)
```

这会非常慢。更合理的做法，是只关注当前序列中真实出现的 `pair`，并使用优先队列、链表或增量更新计数等方式加速 `merge` 过程。Assignment 1 中通常就会要求你思考如何只对“可能发生变化的局部”做更新，而不是全局反复扫描。

(4) **正确处理 special tokens。** 真实 tokenizer 通常需要保留诸如 `<endoftext>|>|` 之类的特殊 token，它们不应被普通 BPE 规则随意拆开或重新合并。因此，工程上通常要在预处理阶段显式识别并保护这些特殊符号。

(5) **预切分策略非常重要。** 如果完全不做 pre-tokenization，BPE 学到的 merges 可能跨越本不应跨越的边界，影响压缩效果和语义稳定性。因此，合理设计预切分规则是 tokenizer 质量的重要组成部分。

(6) **为调试保留可解释性。** 学习阶段建议多打印：原始字符串、字节序列、初始 token、每轮 merge 后的序列、最终 decode 结果。Tokenizer 出错时，最有效的调试方式往往不是盲目改代码，而是逐层检查表示是否仍然一致。

2.13 Conceptual Summary

从更高层次看，tokenization 可以被理解作为一种**接口设计与压缩设计的结合**。它一方面要在“字符串世界”和“整数序列世界”之间建立可逆接口，另一方面又要通过统计规律尽可能压缩高频结构，从而减小后续模型的序列处理成本。字符级方案过细，字节级方案过长，单词级方案过粗，而 BPE 提供了一个在实际工程中非常成功的折中。

因此，本章的核心结论并不是“BPE 是唯一正确的方法”，而是：在当前 Transformer 主导的大模型体系中，tokenization 的优劣会直接影响序列长度、训练效率、模型容量利用率以及实现复杂度。也正因为如此，tokenization 成为这门课进入具体实现之后首先讨论的主题。

2.14 Summary

本章围绕 tokenization 建立了三个层面的理解。首先，从系统视角看，tokenizer 是字符串与模型输入之间不可缺少的中间层；其次，从方法比较看，字符级、字节级和单词级方案各有优势，但都存在明显局限，而 BPE 是一种更实用的折中；最后，从实现视角看，教学代码虽然简洁，却已经展示了 BPE 的核心机制：统计高频相邻对、反复 merge、构造可逆 vocabulary，并在 encode/decode 中复用这些规则。理解了这些内容之后，学生就真正具备了从零实现一个基础 tokenizer 的起点。

3 Lecture 02, Part I: Motivation, Tensor Basics, and Memory Accounting

3.1 Overview of This Part

本讲是 CS336 第二讲的第一部分，对应源码中从 `main()` 开始到 `tensors_memory()` 为止的内容。其核心目标不是讲 Transformer 的具体结构，而是建立训练模型所需的最基础观念：

- 训练模型时我们究竟在消耗什么资源；
- 为什么需要做 *memory accounting* 与 *compute accounting*；
- PyTorch 中的 tensor 是什么；
- 不同数值精度（fp32 / fp16 / bf16 / fp8）在内存与数值稳定性上的差异。

lecture 源码一开始明确给出本讲的定位：

We will discuss all the primitives needed to train a model.

We will go bottom-up from tensors to models to optimizers to the training loop.

We will pay close attention to efficiency (use of resources).

这里的“resources”在这一讲中特别指两类：

- **Memory**（显存 / 内存），单位通常是 GB；
- **Compute**（计算量），单位通常是 FLOPs。

因此，这一部分的真正主题可以概括为：

训练模型之前，必须先学会做资源核算。

3.2 Motivating Questions: Why Resource Accounting Matters

lecture 的第一部分先通过两个 *napkin math*（餐巾纸级估算）问题建立直觉。

3.2.1 Question 1: Training a 70B model on 15T tokens with 1024 H100s

源码中给出：

```
total_flops = 6 * 70e9 * 15e12
assert h100_flop_per_sec == 1979e12 / 2
mfu = 0.5
flops_per_day = h100_flop_per_sec * mfu * 1024 * 60 * 60 * 24
days = total_flops / flops_per_day
```

这段代码使用了一个极其重要的经验公式：

$$\text{training FLOPs} \approx 6 \times N \times D,$$

其中：

- N 表示模型参数量;
- D 表示训练 token 数量。

对题目中的条件:

$$N = 70 \times 10^9, \quad D = 15 \times 10^{12},$$

因此总 FLOPs 为:

$$6 \cdot 70 \times 10^9 \cdot 15 \times 10^{12} = 6.3 \times 10^{24}.$$

然后 lecture 假设单张 H100 的 dense 峰值吞吐约为:

$$1979 \text{ teraFLOP/s/2},$$

即因为 1979 teraFLOP/s 是带 sparsity 的数值, 所以 dense 情况下取其一半。

进一步, lecture 假设:

$$\text{MFU} = 0.5,$$

即模型 FLOPs 利用率只有理论峰值的 50%。这已经是一个相当不错但仍不理想化的估计。

因此, 1024 张 H100 每天的有效 FLOPs 大致是:

$$\text{flops per day} = (\text{single-GPU FLOP/s}) \times 0.5 \times 1024 \times 86400.$$

最终,

$$\text{days} = \frac{\text{total FLOPs}}{\text{effective FLOPs per day}}.$$

理解与注意事项 这一题的作用不是为了得到一个绝对精确的天数, 而是建立一个关键直觉:

训练大模型时, 数量级估算本身就是研究与工程能力的一部分。

这里需要特别注意三点:

1. **6ND** 是一个常用的一阶近似, 不是精确公式;
2. 使用的是 **有效吞吐**, 而不是理论峰值吞吐;
3. 真实训练中还会受到通信、调度、I/O、内核效率、激活重算等因素影响, 因此结果只能看作粗略估算。

3.2.2 Question 2: How large a model can 8 H100s hold with AdamW (naively)?

lecture 中接着给出:

```
h100_bytes = 80e9
bytes_per_parameter = 4 + 4 + (4 + 4)
num_parameters = (h100_bytes * 8) / bytes_per_parameter
```

这段代码在做一个纯内存视角的上界估算。

每个参数为什么要占:

$$4 + 4 + (4 + 4) = 16 \text{ bytes}$$

呢? lecture 中解释为:

- 参数本身: 4 bytes (float32)
- 梯度: 4 bytes (float32)
- AdamW 优化器状态: m 和 v , 各 4 bytes

所以:

$$\text{bytes per parameter} = 16.$$

如果每张 H100 是 80GB, 8 张卡总容量约为:

$$8 \times 80 \times 10^9 \text{ bytes}.$$

于是参数量上界大致为:

$$\# \text{params} \approx \frac{8 \cdot 80 \times 10^9}{16}.$$

lecture 中的两个 caveat 源码紧接着给出两个 caveat:

1. 这里假设参数和梯度都用 float32。实际中也可能使用 bf16 参数和梯度, 再额外保留一份 float32 master weights。lecture 指出这并不一定省内存, 但通常更快。
2. 这里完全没有计算 activation memory, 而 activation memory 与 batch size、sequence length 强相关。

理解与注意事项 这一题的重点不是算出精确可训练的参数上限, 而是告诉我们:

模型能否训练, 第一关往往不是 FLOPs, 而是显存能否放下参数、梯度、优化器状态和激活。

进一步说, 这里只是最 naive 的 data-parallel 视角。真实训练中, 像 ZeRO、FSDP、张量并行等方法都会改变显存分布, 因此这个结果更像是:

“不做高级并行优化时的粗略内存上界。”

3.3 Tensor Basics

在做完两个 motivating questions 之后, lecture 开始进入最基础的对象: **tensor**。源码中写道:

Tensors are the basic building block for storing everything: parameters, gradients, optimizer state, data, activations.

这句话非常重要。在深度学习训练中, 几乎一切都以 tensor 的形式存在:

- 参数 (parameters) 是 tensor;
- 梯度 (gradients) 是 tensor;
- 优化器状态 (optimizer states) 是 tensor;

- 输入数据 (data / batch) 是 tensor;
- 前向传播中间结果 (activations) 也是 tensor。

所以如果不理解 tensor, 就不可能真正理解训练系统。

3.3.1 How tensors are created

lecture 中给出几种最基本的创建方式:

```
x = torch.tensor([[1., 2, 3], [4, 5, 6]])
x = torch.zeros(4, 8)
x = torch.ones(4, 8)
x = torch.randn(4, 8)
x = torch.empty(4, 8)
nn.init.trunc_normal_(x, mean=0, std=1, a=-2, b=2)
```

其含义分别是:

- `torch.tensor(...)`: 根据已有值直接构造 tensor;
- `torch.zeros`: 全零初始化;
- `torch.ones`: 全一初始化;
- `torch.randn`: 用标准正态分布随机初始化;
- `torch.empty`: 只分配内存, 不初始化数值;
- `nn.init.trunc_normal_`: 使用截断正态分布进行初始化。

理解与注意事项 这里有一个很容易忽视的点: `torch.empty` 并不是“全 0”, 而是未初始化的内存。因此其中的值可能看起来是随机垃圾值。使用它的理由是: 如果你反正马上会用自己的初始化逻辑覆盖这些值, 那么就没必要先把内存清零, 从而节省初始化成本。

因此:

`empty` 的语义不是“没有值”, 而是“内存已分配, 但内容未定义”。

3.4 Tensor Memory Accounting

这是第一部分最核心的内容之一: **tensor 的内存占用如何计算?**

lecture 先给出核心原则:

Memory is determined by the (i) number of values and (ii) data type of each value.

也就是说:

$$\text{memory usage} = (\text{number of elements}) \times (\text{bytes per element}).$$

源码中通过以下属性做说明:

```
x = torch.zeros(4, 8)
assert x.dtype == torch.float32
assert x.numel() == 4 * 8
assert x.element_size() == 4
assert get_memory_usage(x) == 4 * 8 * 4
```

其中:

- `x.numel()`: tensor 中总元素个数;
- `x.element_size()`: 每个元素占多少字节;
- `get_memory_usage(x)`: 源码中定义为

```
def get_memory_usage(x: torch.Tensor):
    return x.numel() * x.element_size()
```

3.4.1 float32

lecture 首先介绍 float32。

- 默认 tensor dtype 通常是 `torch.float32`;
- 每个元素占 4 bytes;
- 它是传统科学计算中的基本精度。

例如一个 4×8 的 fp32 tensor 占用:

$$4 \times 8 \times 4 = 128 \text{ bytes.}$$

lecture 还举了一个非常有冲击力的例子:

```
assert get_memory_usage(torch.empty(12288 * 4, 12288)) == 2304 * 1024 * 1024
```

这个矩阵的大小是:

$$(12288 \times 4) \times 12288,$$

对应 GPT-3 前馈层中的一张权重矩阵, 光这一张矩阵就大约要:

$$2.3 \text{ GB.}$$

理解与注意事项 这说明:

在大模型里, 单个权重矩阵本身就可能非常大, 参数内存绝不是一个可以忽略的量。

因此训练大模型时, “一层有多大” 不只是结构问题, 也是显存问题。

3.4.2 float16

lecture 接着介绍 float16:

- 每个元素占 2 bytes;
- 相比 float32, 内存减半;
- 但数值动态范围较差, 尤其是小数很容易 underflow。

源码中的例子:

```
x = torch.tensor([1e-8], dtype=torch.float16)
assert x == 0
```

这说明 10^{-8} 在 fp16 中已经小到无法表示, 从而直接下溢为 0。

理解与注意事项 这件事在训练中是很危险的, 因为梯度、方差、归一化统计量中经常会出现很小的数。一旦这些值下溢为 0, 就可能导致:

- 梯度消失;
- 更新不稳定;
- 训练发散或停滞。

所以 lecture 的结论是: 虽然 fp16 更省内存, 但训练时有明显稳定性风险。

3.4.3 bfloat16

lecture 随后引出 bfloat16:

- 仍然只占 2 bytes;
- 但指数位与 float32 接近, 因此动态范围与 float32 类似;
- 虽然精度分辨率更差, 但通常更适合深度学习训练。

源码中的对比例子:

```
x = torch.tensor([1e-8], dtype=torch.bfloat16)
assert x != 0
```

这说明同样是 10^{-8} , 在 bf16 中不会直接下溢为 0。

理解与注意事项 这就是为什么现代大模型训练中, bf16 非常重要。它几乎可以看作是:

“保留 fp16 内存优势, 同时尽量接近 fp32 数值范围的折中方案。”

因此在工程上, bf16 往往比 fp16 更适合作为训练主力低精度格式。

3.4.4 fp8

lecture 最后简要提到 fp8:

- 2022 年标准化;
- H100 支持两种 FP8 格式;
- 更低内存、更高吞吐, 但稳定性风险更高。

lecture 的态度不是说 fp8 不能用, 而是强调:

精度越低, 内存和速度越好, 但训练稳定性风险越大。

因此在本讲后面才会引出 mixed precision training: 不是全部都用低精度, 而是选择性地在合适位置使用低精度。

3.5 What This Part Really Wants Us to Learn

从学习角度看, 这一部分最想传达的其实不是某个 PyTorch API, 而是以下四点。

3.5.1 (1) Always do rough resource accounting first

在开始训练任何模型之前, 都应该先问:

- 需要多少 FLOPs?
- 需要多少显存?
- 多少张 GPU?
- 大概要跑多久?

也就是说, 先做数量级估算, 再考虑实现细节。

3.5.2 (2) Tensor is the universal storage unit

在训练系统里, 几乎所有核心对象都归结为 tensor。因此理解 tensor 的 dtype、shape、元素个数与内存占用, 是一切后续分析的起点。

3.5.3 (3) Memory accounting is simple in form, but powerful in effect

公式

$$\text{memory} = \text{numel} \times \text{element size}$$

看起来简单, 但它能帮助我们快速判断:

- 一个模型能否放入显存;
- 参数、梯度、优化器状态谁更占地方;
- 为什么低精度训练能显著改变可训练规模。

3.5.4 (4) Precision choice is an engineering tradeoff

不存在一种精度在所有维度都完美。必须权衡：

- float32: 稳定，但贵；
- float16: 省，但风险较高；
- bfloat16: 现代训练中常见折中；
- fp8: 更激进，需要更强的工程支持。

3.6 Summary Notes

这一部分可以总结为以下学习笔记要点：

1. 训练模型首先要做资源核算，特别是 **memory** 和 **compute**;
2. 一个常用的一阶训练 FLOPs 近似是

$$6 \times (\#\text{parameters}) \times (\#\text{tokens});$$

3. naive AdamW 训练中，每个参数往往不只占参数本身的内存，还要考虑梯度和优化器状态；
4. tensor 是训练系统中参数、梯度、数据、激活、优化器状态的统一表示；
5. tensor 内存占用由元素总数与每元素字节数共同决定；
6. fp32 稳定但昂贵，fp16 节省但存在明显下溢风险，bf16 是更适合训练的低精度折中方案；
7. 在大模型训练中，单个权重矩阵就可能达到 GB 级，因此显存核算必须成为日常思维习惯。

4 Lecture 02, Part II: Tensors on GPUs and Basic Tensor Operations

4.1 Overview of This Part

这一部分对应 `lecture_02.py` 中以下内容：

- `tensors_on_gpus()`
- `tensor_operations()`
- `tensor_storage()`
- `tensor_slicing()`
- `tensor_elementwise()`
- `tensor_matmul()`

这一部分的主线是：在第一部分已经建立了“tensor 是训练系统中的基本存储单位”这一观念之后，现在进一步讨论：

1. tensor 默认放在哪里；
2. 为什么训练时必须把 tensor 放到 GPU 上；
3. tensor 在 PyTorch 中的内部表示是什么；
4. 哪些操作只是 view，哪些操作会产生真正的内存拷贝；
5. 为什么矩阵乘法是深度学习中的核心运算。

因此，这一部分的真正主题可以概括为：

理解 tensor 不仅是理解数学数组，更是理解它在内存中的存储方式以及它在 GPU 上如何被高效操作。

4.2 Tensors on GPUs

4.2.1 Default device: CPU

lecture 首先说明：

By default, tensors are stored in CPU memory.

对应源码：

```
x = torch.zeros(32, 32)
assert x.device == torch.device("cpu")
```

这说明，在 PyTorch 中如果你不显式指定 device，那么新建的 tensor 默认会放在 CPU 内存中。

理解与注意事项 这件事虽然简单，但非常重要，因为很多初学者会误以为“我有 GPU，所以 PyTorch 会自动用 GPU”。实际上不是这样：

- 有 GPU 只是说明你 可以使用 GPU；
- 但 tensor 是否真的在 GPU 上，取决于其 device；
- 如果模型在 GPU 上而数据在 CPU 上，或者反过来，计算会直接报错。

因此，一个非常基本的习惯是：任何时候都明确检查 tensor 和 model 在哪个 device 上。

4.2.2 Why move tensors to GPU?

lecture 接着说：

However, in order to take advantage of the massive parallelism of GPUs, we need to move them to GPU memory.

也就是说，深度学习训练之所以依赖 GPU，并不是因为 GPU “更高级”，而是因为：

- GPU 拥有更高的并行度；
- 对矩阵乘法、卷积、张量操作等具有极高吞吐；
- 深度学习的大多数核心计算都正好适合 GPU 的并行计算模式。

源码中还先检查是否有 GPU：

```
if not torch.cuda.is_available():
    return
num_gpus = torch.cuda.device_count()
for i in range(num_gpus):
    properties = torch.cuda.get_device_properties(i)
```

这里的逻辑是：

- 如果 CUDA 不可用，那么这一段示例就直接结束；
- 如果 CUDA 可用，则查询 GPU 数量与每张卡的属性。

理解与注意事项 这里传达出一个很真实的工程习惯：代码要能在没有 GPU 的环境中优雅退化到 CPU。这也是为什么 lecture 中很多函数都先写：

```
if not torch.cuda.is_available():
    return
```

这种写法让示例代码在课堂、笔记本、本地 CPU 环境中也能运行。

4.2.3 Moving tensors to GPU

lecture 随后给出最基本的迁移方式:

```
y = x.to("cuda:0")
assert y.device == torch.device("cuda", 0)
```

这表示将 CPU tensor x 复制到 GPU 0 上, 并得到一个新的 tensor y 。
还给出另一种方式:

```
z = torch.zeros(32, 32, device="cuda:0")
```

这表示直接在 GPU 上创建 tensor。

理解与注意事项 这两种方式的差别是:

- `x.to("cuda:0")`: 先在 CPU 上有 tensor, 再复制到 GPU;
- `torch.zeros(..., device="cuda:0")`: 直接在 GPU 上分配并初始化。

从工程角度看, 后一种方式往往更自然, 也避免了一次不必要的 CPU→GPU 拷贝。

4.2.4 Memory accounting when moving to GPU

源码中进一步观察 GPU 显存占用变化:

```
memory_allocated = torch.cuda.memory_allocated()
y = x.to("cuda:0")
z = torch.zeros(32, 32, device="cuda:0")
new_memory_allocated = torch.cuda.memory_allocated()
memory_used = new_memory_allocated - memory_allocated
assert memory_used == 2 * (32 * 32 * 4)
```

这里说明:

- 创建了两个 32×32 的 float32 矩阵;
- 每个矩阵的内存占用是

$$32 \times 32 \times 4 \text{ bytes};$$

- 两个矩阵总计正好是两倍这个值。

理解与注意事项 这一步很重要, 因为它把第一部分的 memory accounting 公式真正落实到了 GPU 上。也就是说, 显存核算并不是抽象数学, 而是可以直接通过 API 观察到的实际变化。

需要注意的是, 真实 PyTorch/CUDA 环境中, 显存管理器可能会缓存内存, 因此并非所有情况下 `memory_allocated` 的变化都绝对等于你手算的值。但 lecture 这里用的是一个足够小且清晰的例子, 用来建立最基本的直觉。

4.3 Tensor Operations

lecture 接着说:

Most tensors are created from performing operations on other tensors.

Each operation has some memory and compute consequence.

这是这一部分的核心转折点。

也就是说，在真实训练中，tensor 很少只是被“创建”，更多时候它们是在别的 tensor 上操作得到的。于是问题就变成：

- 一个操作是否分配新内存？
- 一个操作是否只是 view？
- 一个操作的计算代价大不大？

lecture 把这一部分分为四块：

- tensor storage
- tensor slicing
- tensor elementwise operations
- tensor matrix multiplication

4.4 Tensor Storage

4.4.1 What is a tensor in PyTorch?

lecture 中非常关键的一句话是：

PyTorch tensors are pointers into allocated memory ... with metadata describing how to get to any element of the tensor.

这句话意味着：在 PyTorch 中，tensor 并不只是一个“数学上的矩阵”，而更准确地说，它是：

1. 指向某块底层 storage 的指针；
2. 再加上一组元数据（shape, stride, dtype, device 等）；
3. 这些元数据告诉 PyTorch 如何解释这块存储中的元素。

这是理解 view、transpose、slice、contiguous 的基础。

4.4.2 Stride

lecture 用一个 4×4 的例子来解释 stride:

```
x = torch.tensor([
    [0., 1, 2, 3],
    [4, 5, 6, 7],
    [8, 9, 10, 11],
    [12, 13, 14, 15],
])
assert x.stride(0) == 4
assert x.stride(1) == 1
```

这里:

- $x.stride(0) = 4$ 表示: 沿第 0 维 (行方向) 向前走一格, 需要在底层 storage 中跳过 4 个元素;
- $x.stride(1) = 1$ 表示: 沿第 1 维 (列方向) 向前走一格, 只需跳过 1 个元素。

lecture 还展示了如何利用 stride 找某个元素位置:

```
r, c = 1, 2
index = r * x.stride(0) + c * x.stride(1)
assert index == 6
```

即:

$$\text{index} = r \cdot \text{stride}(0) + c \cdot \text{stride}(1).$$

对于 $(r, c) = (1, 2)$, 就得到:

$$1 \cdot 4 + 2 \cdot 1 = 6.$$

理解与注意事项 这说明: **tensor** 的二维索引本质上最终会被翻译成底层一维 **storage** 的偏移。

一旦你真正理解 stride, 就能理解:

- 为什么 transpose 不一定拷贝内存;
- 为什么某些 tensor 是 non-contiguous;
- 为什么 view 对 contiguous 有要求。

4.5 Tensor Slicing and Views

这一部分是 lecture 中非常值得反复理解的内容。

4.5.1 Views do not copy memory

lecture 中先给出一句核心话:

Many operations simply provide a different **view** of the tensor.

This does not make a copy, and therefore mutations in one tensor affects the other.

然后给出多个例子:

(1) Get row 0

```
x = torch.tensor([[1., 2, 3], [4, 5, 6]])
y = x[0]
assert same_storage(x, y)
```

(2) Get column 1

```
y = x[:, 1]
assert same_storage(x, y)
```

(3) View 2x3 matrix as 3x2 matrix

```
y = x.view(3, 2)
assert same_storage(x, y)
```

(4) Transpose the matrix

```
y = x.transpose(1, 0)
assert same_storage(x, y)
```

这里的辅助函数是:

```
def same_storage(x: torch.Tensor, y: torch.Tensor):
    return x.untyped_storage().data_ptr() == y.untyped_storage().data_ptr()
```

它检查两个 tensor 是否共享同一个底层 storage。

理解与注意事项 这组例子共同说明:

row slicing、column slicing、view、transpose, 在这些例子里都没有复制数据, 而只是换了一种解释同一块内存的方式。

这意味着:

- 这些操作通常非常便宜;
- 它们几乎不增加额外显存;
- 修改一个 view, 原 tensor 也会被修改。

lecture 紧接着验证了这一点：

```
x[0][0] = 100
assert y[0][0] == 100
```

也就是说，x 和 y 指向的是同一份 storage。

4.5.2 Non-contiguous tensors

lecture 进一步指出：

Note that some views are non-contiguous entries, which means that further views aren't possible.

对应例子：

```
x = torch.tensor([[1., 2, 3], [4, 5, 6]])
y = x.transpose(1, 0)
assert not y.is_contiguous()
```

这里 `transpose` 得到的是一个非连续存储的 tensor。为什么？因为转置之后，逻辑上的相邻元素在底层 storage 中不再连续排列。

因此，如果继续执行：

```
y.view(2, 3)
```

会报错。

lecture 中用 `try/except` 展示这一点，并指出错误信息中包含：

```
view size is not compatible with input tensor's size and stride
```

理解与注意事项 这说明：

view 并不是“随便换 shape”都可以，它要求当前 tensor 的底层存储布局能支持这种新视图。

如果 tensor 是 non-contiguous 的，那么仅仅修改 metadata 已经不足以表达目标形状，这时必须先做一次真正的拷贝。

4.5.3 contiguous() creates a copy

lecture 因此给出：

```
y = x.transpose(1, 0).contiguous().view(2, 3)
assert not same_storage(x, y)
```

这说明：

- `contiguous()` 会生成一份连续存储的新 tensor；

- 然后这个新 tensor 才能安全地 view;
- 因为发生了真正拷贝, 所以新 tensor 不再和原 tensor 共用 storage。

lecture 的总结是:

Views are free, copying take both (additional) memory and compute.

理解与注意事项 这是非常关键的工程结论。在模型实现中, 如果你频繁调用:

- `contiguous()`
- `clone()`
- 其他导致拷贝的操作

那么你可能在不知不觉中增加:

- 显存占用;
- 额外计算开销;
- CUDA kernel 调用次数。

因此, 理解 “什么是 view, 什么是 copy” 是性能优化中的基本功。

4.6 Elementwise Operations

lecture 接着介绍逐元素操作:

These operations apply some operation to each element of the tensor ... and return a (new) tensor of the same shape.

源码中给出:

```
x = torch.tensor([1, 4, 9])
assert torch.equal(x.pow(2), torch.tensor([1, 16, 81]))
assert torch.equal(x.sqrt(), torch.tensor([1, 2, 3]))
assert torch.equal(x.rsqrt(), torch.tensor([1, 1 / 2, 1 / 3]))
assert torch.equal(x + x, torch.tensor([2, 8, 18]))
assert torch.equal(x * 2, torch.tensor([2, 8, 18]))
assert torch.equal(x / 0.5, torch.tensor([2, 8, 18]))
```

这些都是典型的逐元素运算: 对每个元素独立作用, 不涉及复杂的跨元素耦合。

lecture 还给了一个矩阵上的例子:

```
x = torch.ones(3, 3).triu()
```

得到上三角矩阵:

$$\begin{bmatrix} 1 & 1 & 1 \\ 0 & 1 & 1 \\ 0 & 0 & 1 \end{bmatrix}$$

并说明它可用于构造 causal attention mask。

理解与注意事项 逐元素操作的特点是：

- 逻辑简单；
- 计算量与元素个数线性相关；
- 通常远比矩阵乘法便宜。

这一点很重要，因为后面在 FLOPs accounting 中，lecture 会明确指出：只要矩阵足够大，深度学习中的主要计算开销通常都来自 **matmul**，而不是 **elementwise operation**。

4.7 Matrix Multiplication

lecture 将矩阵乘法称为：

the bread and butter of deep learning

这句话非常准确。因为在绝大多数神经网络中，最主要的计算都可以归结为矩阵乘法：

- Linear layer
- Attention score computation
- Projection matrices
- MLP layers

4.7.1 Basic example

源码先给出一个最基本的例子：

```
x = torch.ones(16, 32)
w = torch.ones(32, 2)
y = x @ w
assert y.size() == torch.Size([16, 2])
```

其数学形式就是：

$$(16 \times 32) \cdot (32 \times 2) \rightarrow (16 \times 2).$$

4.7.2 Batched matrix multiplication

lecture 接着说明：

In general, we perform operations for every example in a batch and token in a sequence.

因此给出高维示例：

```
x = torch.ones(4, 8, 16, 32)
w = torch.ones(32, 2)
y = x @ w
assert y.size() == torch.Size([4, 8, 16, 2])
```


这里可以这样理解：

- 前面三维 (4, 8, 16) 都是“批处理维度”；
- 最后一维 32 是被乘的输入维度；
- w 的形状是 (32, 2)；
- PyTorch 会自动在前面的批处理维度上广播并逐位置执行矩阵乘法。

因此输出形状为：

(4, 8, 16, 2).

理解与注意事项 这一点在 Transformer 中极其常见，因为实际计算常常不是简单的二维矩阵乘法，而是：

- batch 维度
- sequence 维度
- head 维度
- hidden 维度

叠在一起的 batched matmul。

因此，lecture 这里虽然没有讲 Transformer 本身，但已经在用最基础的例子帮你建立对高维张量乘法的直觉。

4.8 What This Part Really Wants Us to Learn

从学习角度看，这一部分真正想让我们掌握的是以下几点。

4.8.1 (1) Device matters

tensor 不只是有 shape 和 dtype，还有 device。默认在 CPU 上，而训练时通常必须显式放到 GPU 上。如果 device 不一致，计算就会失败。

4.8.2 (2) Tensor is storage plus metadata

PyTorch tensor 的本质不是“数组值”本身，而是：

storage + metadata.

其中 metadata 决定如何读取 storage。这正是 stride、view、transpose 能成立的根本原因。

4.8.3 (3) View is cheap, copy is expensive

切片、转置、reshape/view 在很多情况下并不复制数据，因此成本很低；但一旦需要 contiguous() 或真实拷贝，就会消耗额外内存和计算。

4.8.4 (4) Matrix multiplication dominates

elementwise operation 虽然常见,但真正重的核心计算几乎总是 matmul。这是后续做 FLOPs 分析与性能优化的关键前提。

4.9 Summary Notes

这一部分可以总结为以下笔记要点:

1. PyTorch tensor 默认位于 CPU, 训练时通常需要显式迁移到 GPU;
2. 可以通过 `x.to("cuda:0")` 迁移 tensor, 也可以直接在 GPU 上创建 tensor;
3. tensor 的显存占用仍然服从

$$\text{memory} = \text{numel} \times \text{element size};$$

4. PyTorch tensor 本质上是指向底层 storage 的指针加上一组 metadata;
5. stride 描述沿某个维度移动一步时, 在底层 storage 中需要跳过多少元素;
6. 很多 slicing / transpose / view 操作并不复制数据, 而只是创建 view;
7. view 很便宜, 但 copy 会消耗额外显存和计算;
8. non-contiguous tensor 往往不能直接 view, 需要先 `contiguous()`;
9. elementwise operations 的开销通常是线性的;
10. 深度学习中最核心、最昂贵的操作通常是矩阵乘法, 且它经常以 batched matmul 的形式出现。

5 Lecture 02, Part III: Tensor Dimensions, Jaxtyping, and Einops

5.1 Overview of This Part

这一部分对应 lecture_02.py 中以下内容：

- `tensor_einops()`
- `einops_motivation()`
- `jaxtyping_basics()`
- `einops_einsum()`
- `einops_reduce()`
- `einops_rearrange()`

这一部分在整节课中的位置非常重要。前一部分已经讨论了：

- `tensor` 默认在哪里存储；
- `tensor` 的 `storage` 与 `stride`；
- `slicing` / `view` / `transpose` 的含义；
- `elementwise operation` 与 `matmul` 的基本形式。

但是，一旦 `tensor` 的维度开始变多，例如出现：

`batch, sequence, heads, hidden,`

普通的 PyTorch 写法就会变得越来越难读，也越来越容易写错。于是 lecture 在这一部分引入了两个工具：

1. **jaxtyping**: 给 `tensor` 维度加上名字，增强可读性；
2. **einops**: 用带名字的维度表达张量运算、约简与重排。

因此，这一部分的主线可以概括为：

当 `tensor` 维度变复杂时，清晰地表达维度含义本身就是深度学习实现中的核心能力。

5.2 Motivation: Why Ordinary PyTorch Code Becomes Hard to Read

lecture 先通过一个非常简单的例子说明问题：

```
x = torch.ones(2, 2, 3) # batch, sequence, hidden
y = torch.ones(2, 2, 3) # batch, sequence, hidden
z = x @ y.transpose(-2, -1) # batch, sequence, sequence
```

这里数学上没有问题。如果把形状写出来：

$$x \in \mathbb{R}^{2 \times 2 \times 3}, \quad y \in \mathbb{R}^{2 \times 2 \times 3},$$

做

$$x @ y^{\top}$$

时，实际上是在最后两个维度上进行矩阵乘法，因此输出形状是：

$$z \in \mathbb{R}^{2 \times 2 \times 2}.$$

lecture 对这种写法的评价是：

Easy to mess up the dimensions (what is -2, -1?)...

理解与注意事项 这句话点出了真实工程中的一个大问题：

很多 `tensor` 代码从数值上能跑，但从人类可读性上已经非常脆弱。

在这个例子里，`-2` 和 `-1` 分别表示什么，其实要靠你脑内记住最后两个维度的语义。如果 `tensor` 维度更多，比如：

(batch, seq, heads, hidden),

那么像 `transpose(-2, -1)` 这样的写法就会越来越不透明。

因此，这一部分想建立的第一层直觉是：

“只依赖整数维度索引的张量代码，很容易写对一次，但很难长期维护。”

5.3 Jaxtyping Basics

为了增强维度表达的可读性，lecture 引入了 `jaxtyping`。

5.3.1 Old way vs new way

源码给出对比：

```
x = torch.ones(2, 2, 1, 3) # batch seq heads hidden
```

以及：

```
x: Float[torch.Tensor, "batch seq heads hidden"] = torch.ones(2, 2, 1, 3)
```

这里第二种写法的意思是：

- 这个对象是一个 `torch.Tensor`；
- 它的 `dtype` 被注释为 `Float`；
- 它的各个维度依次命名为：

batch, seq, heads, hidden.

lecture 特别强调：

Note: this is just documentation (no enforcement).

理解与注意事项 这句话非常重要。也就是说，lecture 这里使用 jaxtyping 的目的主要是：

- 作为代码注释；
- 让读代码的人知道每个维度的语义；
- 减少维度理解上的歧义。

但这里并没有强制运行时检查，不会自动验证：

- 你的 batch 维度是不是真的叫 batch；
- hidden 维度大小是否满足预期；
- 维度排列有没有写错。

所以要把它理解成：

一种更结构化的注释，而不是类型系统意义上的严格保证。

进一步理解 虽然 lecture 说它 “just documentation”，但这种 documentation 的价值很大。因为在深度学习代码中，最容易出错的往往不是语法，而是：

- 哪一维是 sequence；
- 哪一维是 heads；
- 某个 view 之后 hidden 是否仍然在最后一维；
- 这个 matmul 究竟是在哪两个维度上进行的。

因此，给维度命名本身就是一种减少 bug 的方式。

5.4 Einops

lecture 随后引入 einops，并给出一句概括：

Einops is a library for manipulating tensors where dimensions are named.

也就是说，einops 的核心思想不是引入新的数学，而是：

把 **tensor** 的维度语义写进运算表达式里。

lecture 说它受 Einstein summation notation 启发。这意味着：我们不再只通过数字维度位置去推断运算，而是直接在表达式中写出维度名字。

lecture 将这一部分分成三种操作：

- einsum
- reduce
- rearrange

5.5 Einops Einsum

5.5.1 The setup

lecture 先定义两个 tensor:

```
x: Float[torch.Tensor, "batch seq1 hidden"] = torch.ones(2, 3, 4)
```

```
y: Float[torch.Tensor, "batch seq2 hidden"] = torch.ones(2, 3, 4)
```

因此:

$$x \in \mathbb{R}^{2 \times 3 \times 4}, \quad y \in \mathbb{R}^{2 \times 3 \times 4}.$$

这里:

- 第一维是 `batch`;
- 第二维分别是 `seq1` 与 `seq2`;
- 最后一维是 `hidden`。

5.5.2 Old way

lecture 先用普通 PyTorch 写法:

```
z = x @ y.transpose(-2, -1)
```

其输出维度为:

$$(batch, seq1, seq2).$$

这一步本质上是在对 `hidden` 维做求和, 因此:

$$z_{b,i,j} = \sum_h x_{b,i,h} y_{b,j,h}.$$

5.5.3 New (einops) way

lecture 接着给出:

```
z = einsum(x, y, "batch seq1 hidden, batch seq2 hidden -> batch seq1 seq2")
```

这条表达式的含义非常清楚:

- 第一个输入有维度 `batch seq1 hidden`;
- 第二个输入有维度 `batch seq2 hidden`;
- 输出维度是 `batch seq1 seq2`;
- 因为 `hidden` 没出现在输出中, 所以自动对 `hidden` 求和。

lecture 明确写道:

Dimensions that are not named in the output are summed over.

这其实就是 Einstein summation 的核心规则。

数学理解 这一表达式对应：

$$z_{b,i,j} = \sum_h x_{b,i,h} y_{b,j,h}.$$

因此，这就是一个带 batch 维度的相似度计算或 batched matrix multiplication。

5.5.4 Using ...

lecture 还给出一个更通用的写法：

```
z = einsum(x, y, "... seq1 hidden, ... seq2 hidden -> ... seq1 seq2")
```

这里的 ... 表示：

任意数量的前缀维度。

也就是说，这个表达式不再绑定“前面一定只有一个 batch 维”，而是允许任何共享的前缀维度一起广播。

理解与注意事项 这在真实工程里非常有用，因为很多时候 tensor 可能是：

(batch, heads, seq, hidden),

也可能是：

(microbatch, batch, seq, hidden).

如果你只关心最后的 **seq** 和 **hidden** 维度结构，那么 ... 写法可以让表达式更泛化。

但需要注意的是：... 虽然灵活，也会让表达式稍微抽象一些。因此在教学和调试阶段，显式写出维度名字往往更清楚。

5.6 Einops Reduce

lecture 接着介绍对单个 tensor 进行约简。

5.6.1 The setup

源码给出：

```
x: Float[torch.Tensor, "batch seq hidden"] = torch.ones(2, 3, 4)
```

因此：

$$x \in \mathbb{R}^{2 \times 3 \times 4}.$$

5.6.2 Old way

普通 PyTorch 写法：

```
y = x.mean(dim=-1)
```

这里对最后一个维度 **hidden** 求均值，因此输出形状为：

$$y \in \mathbb{R}^{2 \times 3}.$$

5.6.3 New (einops) way

lecture 随后写:

```
y = reduce(x, "... hidden -> ...", "sum")
```

注意这里源码示例用的是 "sum"。因此这条语句的含义是:

- 输入维度是 ... hidden;
- 输出维度是 ...;
- 被去掉的 hidden 维做的是 sum 约简, 而不是 mean。

也就是说:

$$y_{...} = \sum_h x_{...,h}.$$

理解与注意事项 这里需要特别注意 lecture 中的一个细节: 前面 “Old way” 用的是:

```
x.mean(dim=-1)
```

但 “New way” 例子里用的是:

```
reduce(..., "sum")
```

所以这两句在严格数值上并不等价。lecture 的重点显然不是要比较 “mean” 和 “sum”, 而是要说明:

einops 的 reduce 可以用带名字的维度表达各种约简, 而不是只依赖 dim=-1 这种位置索引。

因此, 做学习笔记时不能把这两句强行说成 “完全一样”, 而应如实记录: lecture 这里是在用一个 reduce 示例说明写法, 而不是强调与 mean(dim=-1) 数值完全一致。

5.7 Einops Rearrange

这一部分是 lecture 中最能体现 “维度名字有助于理解” 的部分。

5.7.1 Motivation

lecture 先说:

Sometimes, a dimension represents two dimensions ... and you want to operate on one of them.

这是深度学习中极其常见的现象。例如一个维度 total_hidden, 其实可能是:

heads × hidden1

压平后的结果。

源码给出:


```
x: Float[torch.Tensor, "batch seq total_hidden"] = torch.ones(2, 3, 8)
```

然后说明:

...where `total_hidden` is a flattened representation of `heads * hidden1`

并给出一个矩阵:

```
w: Float[torch.Tensor, "hidden1 hidden2"] = torch.ones(4, 4)
```

5.7.2 Step 1: Split a dimension

lecture 首先用:

```
x = rearrange(x, "... (heads hidden1) -> ... heads hidden1", heads=2)
```

这表示:

- 原来的最后一维 `total_hidden` 被解释为 `heads hidden1` 的乘积;
- 指定 `heads=2`;
- 因为原来 `total_hidden=8`, 所以可推得 `hidden1=4`;
- 因此输出形状变为:

(2, 3, 2, 4).

理解 这一步本质上是在做一个“带语义的 reshape”。普通 PyTorch 里你可能会写:

```
x = x.view(2, 3, 2, 4)
```

但这种写法不会告诉读者:

- 2 是什么;
- 4 是什么;
- 为什么原来的 8 要拆成这两个数。

而 `rearrange` 的写法直接把这一层语义写出来了。

5.7.3 Step 2: Apply a transformation on one sub-dimension

lecture 接着写:

```
x = einsum(x, w, "... hidden1, hidden1 hidden2 -> ... hidden2")
```

这表示:

- 对每个前缀维度位置上的 `hidden1` 向量;
- 都右乘同一个矩阵 `w`;

- 从而把 `hidden1` 变成 `hidden2`。

如果当前

$$x \in \mathbb{R}^{2 \times 3 \times 2 \times 4}, \quad w \in \mathbb{R}^{4 \times 4},$$

那么输出仍然是：

$$x \in \mathbb{R}^{2 \times 3 \times 2 \times 4},$$

只是最后一维从原来的 `hidden1` 空间变换到了 `hidden2` 空间。

理解 这一步其实很像多头结构中的常见做法：先把一个“合并的大维度”拆成多个语义维度，然后只对其中某一维施加线性变换。

5.7.4 Step 3: Merge dimensions back

`lecture` 最后写：

```
x = rearrange(x, "... heads hidden2 -> ... (heads hidden2)")
```

这表示将最后两个维度重新合并成一个维度。如果当前形状是：

$$(2, 3, 2, 4),$$

则输出回到：

$$(2, 3, 8).$$

理解与注意事项 这一整段流程其实展示了一个非常常见的模式：

1. 先把一个压平维度拆开；
2. 在子维度上做操作；
3. 再重新合并回去。

这个模式在 Transformer 中处处可见，例如：

- `hidden` 维拆成 `heads × head_dim`；
- 做 `attention`；
- 再合并 `heads` 与 `head_dim`。

因此，`lecture` 虽然还没有正式进入 Transformer，但这一部分其实已经在训练你以“多维度语义重排”的方式理解深度学习实现。

5.8 What This Part Really Wants Us to Learn

从学习角度看，这一部分真正想传达的是以下几点。

5.8.1 (1) Dimension bookkeeping is a real problem

在小 tensor 上, `transpose(-2,-1)`、`view(...)`、`mean(dim=-1)` 似乎很简单。但一旦维度增多, 就会迅速变得难以维护。因此, 维度管理本身是深度学习工程中的真实难点。

5.8.2 (2) Named dimensions improve readability

`jaxtyping` 与 `einops` 的核心价值不是更快, 而是更清楚。它们通过给维度命名, 让代码更接近数学表达, 也更接近人类理解方式。

5.8.3 (3) Einsum expresses contraction clearly

`einsum` 的关键优势在于:

- 输入维度名字清楚;
- 输出维度名字清楚;
- 哪些维度被求和一眼可见。

因此它特别适合表达 attention、相似度、投影等结构。

5.8.4 (4) Rearrange expresses shape transformation semantically

`rearrange` 的价值不只是 `reshape`, 而是:

把“为什么这么 `reshape`”写进代码本身。

这使得拆分 heads、合并 hidden、重排 batch/sequence 维度等操作更加自然。

5.8.5 (5) Documentation matters even without enforcement

lecture 特别强调 `jaxtyping` 在这里主要是 documentation。这说明在科研与工程中:

好的注释、好的维度命名、好的表达方式, 本身就是减少 bug 的重要手段。

5.9 Summary Notes

这一部分可以总结为以下学习笔记要点:

1. 随着 tensor 维度增多, 仅靠 `dim=-1`、`transpose(-2,-1)` 等位置索引写法会越来越难读;
2. `jaxtyping` 允许用名字注释 tensor 的各个维度, 如 `"batch seq heads hidden"`;
3. 在 lecture 中, `jaxtyping` 的作用主要是 documentation, 而不是运行时强制检查;
4. `einops` 的核心思想是: 用命名维度来表达 tensor 运算;
5. `einsum` 中, 输入里出现但输出里未出现的维度会被求和;
6. `...` 表示任意共享的前缀维度, 有助于写出更通用的表达式;

7. `reduce` 可以清晰表达在某一命名维度上的求和、均值等约简操作；
8. `rearrange` 可以把一个压平维度拆成多个语义维度，也可以把多个维度重新合并；
9. 这类写法对 Transformer 等高维张量结构尤其重要，因为其中经常需要拆分 heads、重排 hidden、处理 batch/sequence 等多重维度。

6 Lecture 02, Part IV: Compute Accounting, FLOPs, and Gradients

6.1 Overview of This Part

这一部分对应 lecture_02.py 中以下内容：

- `tensor_operations_flops()`
- `gradients_basics()`
- `gradients_flops()`

在前面的部分中，lecture 已经完成了两件基础工作：

1. 从 **memory** 角度理解 tensor 与参数存储；
2. 从 **tensor operation** 角度理解 view、slice、elementwise operation 与 matrix multiplication。

而这一部分开始正式进入第二类资源：

compute $\iff$ **FLOPs.**

也就是说，这一部分的核心问题是：

- 一个操作究竟需要多少计算；
- 这些 FLOPs 与 wall-clock time 是什么关系；
- forward pass 和 backward pass 的计算量如何比较；
- 为什么训练语言模型时常用

$$6 \times (\text{\#tokens}) \times (\text{\#parameters})$$

作为一阶 FLOPs 估算。

因此，这一部分的主线可以概括为：

训练的计算代价可以被系统地估算，而 **backward** 通常比 **forward** 更贵。

6.2 What is a FLOP?

lecture 首先给出定义：

A floating-point operation (FLOP) is a basic operation like addition $(x + y)$ or multiplication (xy) .

也就是说，FLOP 是一个**计算量单位**，用来衡量进行了多少次浮点基本运算。

随后 lecture 特别提醒两个很容易混淆的术语：

- **FLOPs**: floating-point operations，表示总共做了多少浮点运算；
- **FLOP/s**（也常写 FLOPS）: floating-point operations per second，表示硬件每秒能做多少浮点运算。

理解与注意事项 这两个缩写看起来几乎一样，但概念完全不同：

- FLOPs 是 **工作量**；
- FLOP/s 是 **速度**。

更形象地说：

- FLOPs 类似于“总共搬了多少箱货”；
- FLOP/s 类似于“每秒能搬多少箱货”。

因此，wall-clock time（训练时间）大致满足：

$$\text{time} \approx \frac{\text{total FLOPs}}{\text{actual FLOP/s}}.$$

6.3 Large-Scale Intuition: Why FLOPs Matter

lecture 先给出几个数量级直觉：

- 训练 GPT-3（2020）大约用了 3.14×10^{23} FLOPs；
- 训练 GPT-4（2023）被推测用了 2×10^{25} FLOPs；
- 还提到美国曾要求训练 FLOPs 超过 10^{26} 的 foundation model 需报告政府（后于 2025 撤销）。

然后 lecture 给出硬件峰值吞吐：

- A100 峰值约为 312 teraFLOP/s；
- H100 在带 sparsity 时约 1979 teraFLOP/s，dense 情况取一半。

源码中也写了：

```
assert a100_flop_per_sec == 312e12
assert h100_flop_per_sec == 1979e12 / 2
```

理解与注意事项 这一步的目的不是背硬件参数，而是建立一个意识：

大模型训练的成本级别极高，因此 FLOPs 估算不是附属技能，而是核心工程能力。

同时还要注意：这些 peak FLOP/s 是理想上限，不等于真实训练时你能达到的吞吐。

6.4 A First-Principles Example: Linear Model FLOPs

为了建立最基本的计算量分析方法，lecture 先从一个线性模型出发。

6.4.1 The setup

源码中设定:

```
if torch.cuda.is_available():
    B = 16384
    D = 32768
    K = 8192
else:
    B = 1024
    D = 256
    K = 64
device = get_device()
x = torch.ones(B, D, device=device)
w = torch.randn(D, K, device=device)
y = x @ w
```

这里:

- B : 数据点数 (可以理解为 batch 中的样本数);
- D : 输入维度;
- K : 输出维度。

矩阵形状为:

$$x \in \mathbb{R}^{B \times D}, \quad w \in \mathbb{R}^{D \times K}, \quad y = xw \in \mathbb{R}^{B \times K}.$$

6.4.2 Counting FLOPs

lecture 解释:

We have one multiplication ($x[i][j] \cdot w[j][k]$) and one addition per (i, j, k) triple.

也就是说, 对每个输出元素 $y[i, k]$, 都要计算:

$$y[i, k] = \sum_{j=1}^D x[i, j]w[j, k].$$

对每个三元组 (i, j, k) , 需要:

- 一次乘法;
- 一次加法。

因此总 FLOPs 为:

$$2 \times B \times D \times K.$$

源码中也写为:

```
actual_num_flops = 2 * B * D * K
```

理解与注意事项 这是本部分最基础也最重要的结论之一：

矩阵乘法 $(m \times n) @ (n \times p)$ 的 FLOPs 量级是 $2mnp$ 。

这在深度学习中几乎无处不在。只要你会数 matmul 的 FLOPs，就已经掌握了绝大多数大头计算的估算方法。

6.4.3 Other operations

lecture 随后总结：

- 对 $m \times n$ 矩阵做逐元素操作，需 $O(mn)$ FLOPs；
- 两个 $m \times n$ 矩阵相加，需要 mn FLOPs；
- 对于足够大的矩阵，深度学习中没有什么操作会像 matmul 那样昂贵。

理解与注意事项 这一步是在建立一个极其重要的工程直觉：

只要模型不是特别小，真正主导训练计算成本的几乎总是矩阵乘法，而不是加法、relu、sqrt 之类的逐元素操作。

所以后续做 FLOPs accounting 时，常常允许我们把许多较小操作忽略掉，只保留 matmul 主项。

6.4.4 Interpretation

lecture 给出解释：

- B 是数据点数量；
- $D \times K$ 是参数量；
- forward pass 的 FLOPs 约等于

$$2 \times (\text{\#tokens}) \times (\text{\#parameters}).$$

这里所谓“tokens”在这个线性模型例子里其实对应“data points”。在语言模型里，一个 token 可以被看成一次参数使用的基本单元，因此这种线性模型分析会自然推广到 Transformer 的一阶估算。

理解 这一点非常关键，它解释了为什么 lecture 一再强调：

$$\text{forward FLOPs} \sim 2 \times (\text{\#tokens}) \times (\text{\#parameters}).$$

因为从最简单的线性层看，这种结构已经出现了。

6.5 From FLOPs to Wall-Clock Time

lecture 接着问:

How do our FLOPs calculations translate to wall-clock time (seconds)?

为此, 源码中定义:

```
actual_time = time_matmul(x, w)
actual_flop_per_sec = actual_num_flops / actual_time
```

辅助函数 `time_matmul` 的逻辑是:

- 若使用 CUDA, 先 `torch.cuda.synchronize()`;
- 多次执行 `a @ b`;
- 每次执行后再次 `synchronize`;
- 取平均时间。

为什么要 `synchronize` 这是一个非常重要的 CUDA 细节。因为 GPU kernel 调用通常是异步的, 如果你不调用 `torch.cuda.synchronize()`, 那么计时可能只测到“发射 kernel”的时间, 而不是 kernel 真正执行完的时间。

所以 lecture 在计时函数中显式同步, 是为了得到更真实的 wall-clock time。

6.5.1 Promised FLOP/s

lecture 然后通过 `get_promised_flop_per_sec(device, dtype)` 读取某类硬件和 dtype 下的理论峰值吞吐。

例如:

- A100 + float32: 19.5×10^{12} FLOP/s;
- A100 + bf16/fp16: 312×10^{12} FLOP/s;
- H100 + float32: 67.5×10^{12} FLOP/s;
- H100 + bf16/fp16: $1979 \times 10^{12}/2$ FLOP/s。

这又说明了另一件很重要的事:

FLOP/s 不只依赖硬件型号, 也依赖 dtype。

这正是为什么低精度训练往往不仅节省内存, 还显著提高吞吐。

6.5.2 MFU: Model FLOPs Utilization

lecture 定义:

Definition: (actual FLOP/s) / (promised FLOP/s) [ignore communication/overhead]

即:

$$\text{MFU} = \frac{\text{actual FLOP/s}}{\text{promised FLOP/s}}.$$

lecture 还指出:

Usually, MFU of ≥ 0.5 is quite good

理解与注意事项 MFU 的意义是:

- 分子: 你真实测出来的吞吐;
- 分母: 硬件理论宣传的峰值吞吐;
- 比值越高, 说明你把硬件吃得越满。

但要特别注意 lecture 自己也提醒: promised FLOPs 往往比较乐观, 因此 MFU 不一定是绝对客观的“利用率”, 更像是一种工程指标。

6.5.3 bfloat16 comparison

lecture 接着把

x, w

都转成 bf16, 然后再次计时:

```
x = x.to(torch.bfloat16)
w = w.to(torch.bfloat16)
bf16_actual_time = time_matmul(x, w)
bf16_actual_flop_per_sec = actual_num_flops / bf16_actual_time
bf16_promised_flop_per_sec = get_promised_flop_per_sec(device, x.dtype)
bf16_mfu = bf16_actual_flop_per_sec / bf16_promised_flop_per_sec
```

lecture 总结说:

Note: comparing bfloat16 to float32, the actual FLOP/s is higher.

理解与注意事项 这里体现了 mixed precision / low precision 的一个核心好处:

较低精度不仅省内存, 还能显著提高矩阵乘法吞吐。

当然, 代价就是数值稳定性风险上升, 所以后面才会引出 mixed precision training 作为折中方案。

6.6 Summary of Tensor Operation FLOPs

lecture 在这一部分最后总结：

- Matrix multiplications dominate: $(2mnp)$ FLOPs;
- FLOP/s depends on hardware and dtype;
- MFU = actual FLOP/s divided by promised FLOP/s.

这实际上给出了后续整个 training FLOPs analysis 的基本框架。

6.7 Gradients Basics

接着 lecture 转入 backward / gradient 计算。

源码开头写道：

So far, we've constructed tensors ... and passed them through operations (forward).
Now, we're going to compute the gradient (backward).

6.7.1 A scalar example

lecture 先给出一个简单模型：

$$y = 0.5(x * w - 5)^2$$

对应源码：

```
x = torch.tensor([1., 2, 3])
w = torch.tensor([1., 1, 1], requires_grad=True)
pred_y = x @ w
loss = 0.5 * (pred_y - 5).pow(2)
loss.backward()
```

这里：

- $x = [1, 2, 3]$;
- $w = [1, 1, 1]$ 且 `requires_grad=True`;
- 因此

$$\text{pred_y} = x \cdot w = 1 + 2 + 3 = 6;$$

- 损失为

$$\frac{1}{2}(6 - 5)^2 = \frac{1}{2}.$$

然后执行：

```
loss.backward()
```

就会触发 PyTorch autograd 自动求导。

源码验证：

```

assert loss.grad is None
assert pred_y.grad is None
assert x.grad is None
assert torch.equal(w.grad, torch.tensor([1, 2, 3]))

```

为什么 $w.grad = [1, 2, 3]$ 因为:

$$L = \frac{1}{2}(x \cdot w - 5)^2.$$

先设

$$u = x \cdot w.$$

则:

$$\frac{\partial L}{\partial u} = u - 5.$$

在这里 $u = 6$, 所以:

$$\frac{\partial L}{\partial u} = 1.$$

又因为

$$u = \sum_i x_i w_i,$$

所以:

$$\frac{\partial u}{\partial w_i} = x_i.$$

链式法则给出:

$$\frac{\partial L}{\partial w_i} = \frac{\partial L}{\partial u} \frac{\partial u}{\partial w_i} = 1 \cdot x_i = x_i.$$

因此:

$$w.grad = [1, 2, 3].$$

理解与注意事项 这段例子的重点不是模型本身, 而是:

- 只有设置了 `requires_grad=True` 的叶子张量才会自动积累梯度;
- 中间变量如 `pred_y` 默认不会保留 `.grad`;
- autograd 背后的核心规则就是链式法则。

6.8 Gradients FLOPs

lecture 接着进入更关键的问题:

How many FLOPs is running the backward pass?

6.8.1 The model

源码中考虑一个两层线性模型：

```
x = torch.ones(B, D, device=device)
w1 = torch.randn(D, D, device=device, requires_grad=True)
w2 = torch.randn(D, K, device=device, requires_grad=True)
h1 = x @ w1
h2 = h1 @ w2
loss = h2.pow(2).mean()
```

对应结构是：

$$x \xrightarrow{w_1} h_1 \xrightarrow{w_2} h_2 \rightarrow \text{loss}.$$

其中：

$$x \in \mathbb{R}^{B \times D}, \quad w_1 \in \mathbb{R}^{D \times D}, \quad w_2 \in \mathbb{R}^{D \times K}, \quad h_1 \in \mathbb{R}^{B \times D}, \quad h_2 \in \mathbb{R}^{B \times K}.$$

6.8.2 Forward FLOPs

lecture 先回顾 forward FLOPs：

```
num_forward_flops = (2 * B * D * D) + (2 * B * D * K)
```

这很好理解：

- $x @ w_1$ 需要 $2BDD$ FLOPs；
- $h_1 @ w_2$ 需要 $2BDK$ FLOPs。

总 forward FLOPs 为：

$$2BDD + 2BDK.$$

6.8.3 Backward for w_2

lecture 聚焦于参数 w_2 ，先写出：

$$w_2.grad[j, k] = \sum_i h_1[i, j] * h_2.grad[i, k]$$

这实际上就是：

$$\frac{\partial L}{\partial w_2} = h_1^\top \frac{\partial L}{\partial h_2}.$$

从维度看：

$$h_1 \in \mathbb{R}^{B \times D}, \quad h_2.grad \in \mathbb{R}^{B \times K},$$

因此：

$$w_2.grad \in \mathbb{R}^{D \times K}.$$

对每个 (i, j, k) 都有一次乘法和一次加法，所以需要：

$$2BDK$$

个 FLOPs。

源码中：

```
num_backward_flops += 2 * B * D * K
```

6.8.4 Backward for h_1

lecture 接着写:

$$h1.grad[i, j] = \sum_k w2[j, k] * h2.grad[i, k]$$

即:

$$\frac{\partial L}{\partial h_1} = \frac{\partial L}{\partial h_2} w_2^\top.$$

同样, 对每个 (i, j, k) 有一次乘法和一次加法, 所以又需要:

$$2BDK$$

个 FLOPs。

源码中:

```
num_backward_flops += 2 * B * D * K
```

6.8.5 Backward for w_1

lecture 说:

Can do it for w_1 ($D \times D$ parameters) as well (though don't need $x.grad$).

源码写:

```
num_backward_flops += (2 + 2) * B * D * D
```

这里的 $(2 + 2)BDD = 4BDD$ 对应:

- 计算 $w_1.grad$: $2BDD$;
- 计算 $x.grad$ 或等价的反向传播到更前一层的中量: $2BDD$ 。

虽然这个例子最终并不需要 $x.grad$ 用于参数更新, 但 backward 图传播时, 这一类代价仍然需要被考虑进整体分析。

6.8.6 Putting it together

lecture 最终总结:

- Forward pass: $2 \times (\text{\#data points}) \times (\text{\#parameters})$ FLOPs
- Backward pass: $4 \times (\text{\#data points}) \times (\text{\#parameters})$ FLOPs
- Total: $6 \times (\text{\#data points}) \times (\text{\#parameters})$ FLOPs

这就是 lecture 前面 motivating question 中使用的经验公式来源。

理解与注意事项 这是本部分最重要的结论：

训练时，backward 的 FLOPs 通常大约是 forward 的两倍，因此总训练 FLOPs 约为 forward 的三倍。

进一步说：

$$\text{total training FLOPs} \approx 6 \times (\#\text{tokens}) \times (\#\text{parameters})$$

本质上来自：

- forward: $2ND$
- backward: $4ND$

这种一阶近似。

当然，这并不是对所有架构、所有实现都完全精确，但它足够适合作为大模型训练成本估算的第一原则。

6.9 What This Part Really Wants Us to Learn

从学习角度看，这一部分真正想传达的是以下几点。

6.9.1 (1) FLOPs can be counted systematically

很多初学者把训练计算量看作一种模糊的“很大”。lecture 这一部分的目的恰恰是说明：它不是模糊的，而是可以从操作级别一项一项数出来的。

6.9.2 (2) Matmul is the dominant compute term

只要矩阵足够大，逐元素操作往往是低阶项；真正主导 FLOPs 的是矩阵乘法。这使得我们可以用相对简单的主项近似来估算大模型训练成本。

6.9.3 (3) Hardware throughput depends on both device and dtype

A100 和 H100 不同，fp32 和 bf16 不同。因此：

$$\text{same FLOPs} \neq \text{same wall-clock time.}$$

dtype 的选择不只是数值稳定性问题，也是吞吐问题。

6.9.4 (4) Backward is expensive

lecture 通过显式链式法则推导说明：

- backward 不是“附送的”；
- 它往往比 forward 更贵；
- 在一阶近似下，backward 大约是 forward 的两倍。

6.9.5 (5) The famous 6ND rule has a real derivation

很多地方直接引用

$$6 \times (\text{\#tokens}) \times (\text{\#parameters})$$

但 lecture 这里非常重要的一点是：它不是凭空出现的经验常数，而是从最基本的线性层 forward/backward FLOPs 推导出来的。

6.10 Summary Notes

这一部分可以总结为以下学习笔记要点：

1. FLOP 表示浮点运算次数，FLOP/s 表示每秒浮点运算吞吐，两者必须严格区分；
2. 对矩阵乘法 $(m \times n) @ (n \times p)$ ，主项 FLOPs 为

$$2mnp;$$

3. 在深度学习中，只要矩阵足够大，计算代价通常主要由 matmul 决定；
4. 实际 wall-clock time 近似满足

$$\text{time} \approx \frac{\text{total FLOPs}}{\text{actual FLOP/s}};$$

5. GPU 峰值吞吐不仅依赖硬件型号，也依赖 dtype；
6. MFU 定义为

$$\text{MFU} = \frac{\text{actual FLOP/s}}{\text{promised FLOP/s}};$$

7. autograd 通过链式法则自动计算梯度，只有 `requires_grad=True` 的叶子 tensor 会自动累积梯度；
8. 对线性模型而言，一阶近似下：

$$\text{forward FLOPs} \approx 2 \times (\text{\#tokens}) \times (\text{\#parameters}),$$

$$\text{backward FLOPs} \approx 4 \times (\text{\#tokens}) \times (\text{\#parameters}),$$

因此

$$\text{total training FLOPs} \approx 6 \times (\text{\#tokens}) \times (\text{\#parameters}).$$

7 Lecture 02, Part V: Parameters, Initialization, and Building a Simple Model

7.1 Overview of This Part

这一部分对应 lecture_02.py 中以下内容：

- `module_parameters()`
- `custom_model()`
- `class Linear(nn.Module)`
- `class Cruncher(nn.Module)`
- `get_num_parameters(model)`

在前面的部分中，lecture 已经完成了两层基础铺垫：

1. 从 **memory** 角度理解 `tensor` 与 `dtype`；
2. 从 **compute** 角度理解 `matmul`、FLOPs 与 `backward` 的代价。

而这一部分开始从 “tensor” 过渡到 “model”。换句话说，前面学的是：

“构件是什么”

这一部分开始回答：

“如何把这些构件组织成一个可以训练的模型”

因此，这一部分的主线可以概括为：

模型本质上是带有结构化组织的可训练参数集合，而参数初始化与模块组织方式会直接影响训练稳定性与可实现性。

7.2 Model Parameters in PyTorch

lecture 首先给出一句非常关键的话：

Model parameters are stored in PyTorch as `nn.Parameter` objects.

对应源码：

```
input_dim = 16384
output_dim = 32
w = nn.Parameter(torch.randn(input_dim, output_dim))
assert isinstance(w, torch.Tensor)
assert type(w.data) == torch.Tensor
```

这里说明两件事：

1. `nn.Parameter` 本质上仍然是 `tensor`；
2. 但它在 `nn.Module` 中具有特殊地位：会被自动注册为模型参数。

理解与注意事项 这一点非常重要。因为在 PyTorch 中，并不是所有 tensor 都会被当作“参数”。只有当它们被包成 `nn.Parameter` 并挂在一个 `nn.Module` 上时，才会：

- 出现在 `model.parameters()` 中；
- 出现在 `state_dict()` 中；
- 被优化器自动更新；
- 在训练 checkpoint 中被保存。

也就是说：

`Parameter` 不是“数值上特殊”的 tensor，而是“语义上会被训练系统追踪”的 tensor。

7.3 Parameter Initialization

这部分是 lecture 中非常关键的内容之一。

7.3.1 Naive initialization and its problem

源码先做如下操作：

```
x = nn.Parameter(torch.randn(input_dim))
output = x @ w
assert output.size() == torch.Size([output_dim])
text(f"Note that each element of `output` scales as sqrt(input_dim): {output[0]}.")
```

这里：

- $x \in \mathbb{R}^{input_dim}$;
- $w \in \mathbb{R}^{input_dim \times output_dim}$;
- 所以

$$output = x^\top w \in \mathbb{R}^{output_dim}.$$

lecture 指出：如果直接让 w 的元素服从标准正态分布，那么输出的每个分量会随着

$$\sqrt{input_dim}$$

增长。

为什么会这样 设：

$$x_i \sim \mathcal{N}(0, 1), \quad w_{ij} \sim \mathcal{N}(0, 1),$$

并假设它们近似独立。则某个输出分量可以写为：

$$y_j = \sum_{i=1}^d x_i w_{ij},$$

其中 $d = \text{input_dim}$ 。

由于每一项 $x_i w_{ij}$ 的方差大致是常数量级，因此和的方差大约正比于 d ：

$$\text{Var}(y_j) \propto d.$$

于是其标准差大约增长为：

$$\sqrt{d}.$$

这正是 lecture 所说：

each element of output scales as $\sqrt{\text{input_dim}}$

理解与注意事项 这件事的后果是非常严重的。如果层数变深，那么每一层输出都可能越来越大，从而导致：

- activation 爆炸；
- gradient 爆炸；
- 训练不稳定；
- loss 直接发散。

因此，初始化不是可有可无的细节，而是决定模型能否稳定训练的第一关。

7.3.2 Rescaling by $1/\sqrt{\text{input_dim}}$

lecture 接着给出解决方法：

```
w = nn.Parameter(torch.randn(input_dim, output_dim) / np.sqrt(input_dim))
output = x @ w
text(f"Now each element of `output` is constant: {output[0]}".)
```

这对应将权重缩放为：

$$w_{ij} \sim \frac{1}{\sqrt{d}} \mathcal{N}(0, 1).$$

这样做之后，某个输出分量的方差从原来的 $O(d)$ 变成：

$$\text{Var}(y_j) \propto d \cdot \frac{1}{d} = O(1).$$

因此输出尺度不再随输入维度 d 增长，而保持在常数量级。

lecture 说：

We want an initialization that is invariant to `input_dim`.

这句话非常关键，因为它点出了初始化设计的核心目标：

让不同层的信号尺度尽量保持稳定，而不是随着宽度变化而爆炸或消失。

7.3.3 Relation to Xavier initialization

lecture 随后指出：

Up to a constant, this is Xavier initialization.

并引用了 Xavier initialization 的经典论文。

理解与注意事项 这里要注意 lecture 的措辞是：

“Up to a constant”

也就是说，lecture 并没有声称它和教科书上的 Xavier 初始化公式完全一模一样，而是说：

“这种按 $1/\sqrt{\text{input_dim}}$ 缩放的思想，与 Xavier 初始化属于同一类稳定方差的思路。”

所以在笔记里应该忠实写出：lecture 是在做一种 Xavier 风格的方差控制，而不是声称这里已经完整覆盖所有 Xavier / Kaiming 初始化的细节。

7.3.4 Truncated normal for extra safety

lecture 最后进一步写：

```
w = nn.Parameter(
    nn.init.trunc_normal_(
        torch.empty(input_dim, output_dim),
        std=1 / np.sqrt(input_dim),
        a=-3,
        b=3
    )
)
```

并解释：

To be extra safe, we truncate the normal distribution to $[-3, 3]$ to avoid any chance of outliers.

理解与注意事项 这一步说明 lecture 不仅关心均值和方差，还关心尾部极端值（outliers）。即使总体方差控制得很好，若初始化中偶尔出现极大值，也可能在前向传播中被放大，导致：

- 某些通道数值异常大；
- 局部梯度异常；
- 训练初期不稳定。

所以 truncated normal 的目的可以理解为：

在维持整体方差规模的同时，避免少数极端样本破坏初始稳定性。

7.4 Building a Custom Model

在完成参数与初始化讨论后，lecture 进入真正的“模型构造”。

7.4.1 A simple custom linear layer

lecture 定义了一个非常简单的 `Linear` 类：

```
class Linear(nn.Module):
    def __init__(self, input_dim: int, output_dim: int):
        super().__init__()
        self.weight = nn.Parameter(
            torch.randn(input_dim, output_dim) / np.sqrt(input_dim)
        )

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        return x @ self.weight
```

这个类非常简洁，其核心就是：

- 继承 `nn.Module`；
- 内部持有一个可训练参数 `weight`；
- 在 `forward` 中执行矩阵乘法。

数学上，它实现的是：

$$x \mapsto xW.$$

理解与注意事项 这里特意没有加入 `bias`。这是一个有意的教学简化，因为 lecture 目前关注的是：

- 参数如何被组织；
- `forward` 如何书写；
- 参数量如何统计；
- 初始化如何影响数值尺度。

所以这里的 `Linear` 是最小可训练线性层，而不是 PyTorch 官方 `nn.Linear` 的完整替代品。

7.4.2 A simple deep linear model: Cruncher

lecture 接着定义：

```

class Cruncher(nn.Module):
    def __init__(self, dim: int, num_layers: int):
        super().__init__()
        self.layers = nn.ModuleList([
            Linear(dim, dim)
            for i in range(num_layers)
        ])
        self.final = Linear(dim, 1)

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        B, D = x.size()
        for layer in self.layers:
            x = layer(x)
        x = self.final(x)
        assert x.size() == torch.Size([B, 1])
        x = x.squeeze(-1)
        assert x.size() == torch.Size([B])
        return x

```

这定义了一个简单的深层线性网络：

$$x \rightarrow \underbrace{Linear(dim, dim) \rightarrow \cdots \rightarrow Linear(dim, dim)}_{\text{num_layers times}} \rightarrow Linear(dim, 1).$$

也就是说：

- 前面有若干个隐藏线性层，每层维度不变；
- 最后一个 head 把 dim 维映射到标量；
- 最终对每个 batch 元素输出一个实数。

理解与注意事项 这仍然不是一个“复杂模型”，而是一个教学载体。lecture 在这里故意用深线性模型，而没有马上讲 Transformer，是因为它想让我们先专注于：

- 参数是如何组织进模块的；
- ModuleList 有什么作用；
- forward 是怎么把层串起来的；
- 输出 shape 是如何被严格控制的。

因此，Cruncher 的重点不在建模能力，而在模型结构表达。

7.4.3 Why ModuleList?

这里 lecture 用的是：

```
self.layers = nn.ModuleList(...)
```

而不是普通 Python list。

理解与注意事项 这又是一个非常关键的 PyTorch 细节。如果你把若干层放进普通 list，那么这些子模块不会被 PyTorch 自动注册，于是：

- `model.parameters()` 可能找不到它们；
- `state_dict()` 可能不包含它们；
- 优化器不会更新这些层的参数。

因此：

所有需要被训练系统追踪的子模块，都必须放在 PyTorch 知道的容器里，例如

`nn.ModuleList`。

7.4.4 The final squeeze

lecture 在 forward 最后写：

```
x = self.final(x)
assert x.size() == torch.Size([B, 1])
x = x.squeeze(-1)
assert x.size() == torch.Size([B])
```

这表示：

- 最终 head 输出的是形状 $(B, 1)$ ；
- 但很多损失函数或训练代码更自然地处理 $(B,)$ ；
- 所以把最后一维长度为 1 的维度挤掉。

理解与注意事项 这反映出 lecture 对 shape 管理非常严格。它没有“凭感觉”认为输出差不多，而是每一步都用 `assert` 检查形状。

这是一个非常值得学习的习惯：

在模型实现阶段，shape bug 往往比数值 bug 更常见，而 `assert` 是最便宜的防御手段。

7.5 Inspecting Model Parameters

lecture 中的 `custom_model()` 做了两件很重要的事。

7.5.1 Parameter sizes via state_dict

源码:

```
param_sizes = [
    (name, param.numel())
    for name, param in model.state_dict().items()
]
assert param_sizes == [
    ("layers.0.weight", D * D),
    ("layers.1.weight", D * D),
    ("final.weight", D),
]
```

这里说明:

- `state_dict()` 返回模型中所有可保存状态;
- 每个参数都有明确的名字;
- 可以逐个数每一层的参数个数。

对当前例子:

- 第一层权重: $D \times D$
- 第二层权重: $D \times D$
- 最后一层权重: $D \times 1 = D$

7.5.2 Total number of parameters

lecture 还调用:

```
num_parameters = get_num_parameters(model)
assert num_parameters == (D * D) + (D * D) + D
```

而 `get_num_parameters` 的定义是:

```
def get_num_parameters(model: nn.Module) -> int:
    return sum(param.numel() for param in model.parameters())
```

这说明模型总参数量就是:

$$\sum_{\theta \in \text{model.parameters}()} \text{\#elements in } \theta.$$

理解与注意事项 这是后续做 memory accounting 和 FLOPs accounting 的基础。因为一旦你能数出：

`#parameters,`

就能进一步估算：

- 参数内存；
- 梯度内存；
- optimizer state 内存；
- 训练 FLOPs 规模。

所以“数参数”不是信息展示，而是训练资源分析的起点。

7.6 Moving the Model to GPU

lecture 接着说：

Remember to move the model to the GPU.

源码：

```
device = get_device()
model = model.to(device)
```

这里和前面对 tensor 的讨论一脉相承：模型内部的参数本质上也是 tensor，因此它们也必须位于正确的 device 上。

理解与注意事项 如果模型在 GPU 上，而输入数据还在 CPU 上，那么

$model(x)$

通常会直接报错。因此真实训练时必须始终保证：

- 模型参数在正确 device；
- 输入 batch 在同一个 device；
- 中间创建的新 tensor 也尽量在同一 device。

7.7 Running the Model

lecture 最后给出一个最基本的调用示例：

```
B = 8
x = torch.randn(B, D, device=device)
y = model(x)
assert y.size() == torch.Size([B])
```

这说明：

- 输入 batch 的形状是 (B, D) ；
- 经过若干层线性变换后；
- 最终输出一个 shape 为 $(B,)$ 的向量。

理解 这里的意义不在于模型多强，而在于：

到这里为止，lecture 已经从“单个 tensor 运算”推进到了“一个真正可运行的、带参数的、可放到 GPU 上的模型”。

这是从 primitives 走向完整训练系统的重要一步。

7.8 What This Part Really Wants Us to Learn

从学习角度看，这一部分真正想传达的是以下几点。

7.8.1 (1) Parameters are special tensors

`nn.Parameter` 本质上仍然是 tensor，但它在模块系统中会被自动注册，因此能被优化器、`state_dict` 和 checkpoint 追踪。

7.8.2 (2) Initialization is about signal scale

初始化的关键不是“随机一点”，而是要控制前向传播的数值尺度，使其不随输入维度变大而爆炸。按 $1/\sqrt{\text{input_dim}}$ 缩放，就是这种思想的一个典型体现。

7.8.3 (3) Models are structured parameter collections

一个模型并不神秘，本质上就是：

- 一组被结构化组织起来的参数；
- 一套在 `forward` 中定义的计算图。

7.8.4 (4) Module registration matters

不是所有 Python 对象里的 tensor 都会自动被训练系统追踪。只有放在 `nn.Module` / `nn.Parameter` / `nn.ModuleList` 等机制里的对象，才真正属于模型。

7.8.5 (5) Shape discipline matters

lecture 中频繁使用 `assert` 检查 shape，这表明：

模型实现的正确性，首先体现在 shape 正确。

7.9 Summary Notes

这一部分可以总结为以下学习笔记要点：

1. PyTorch 中模型参数通过 `nn.Parameter` 表示，它本质上是 `tensor`，但会被模块系统自动注册；
2. 只有被注册的参数才会出现在 `model.parameters()`、`state_dict()` 中，并被优化器更新；
3. 直接用标准正态初始化权重时，输出尺度会随着 $\sqrt{input_dim}$ 增长，这会导致训练不稳定；

4. 用

$$\frac{1}{\sqrt{input_dim}}$$

对初始化进行缩放，可以让输出尺度保持在常数量级；

5. lecture 将这种初始化视为 Xavier 风格初始化的一个版本，并进一步用 truncated normal 抑制 outliers；
6. 一个简单模型可以通过继承 `nn.Module`、组织若干 `Linear` 层并定义 `forward` 来实现；
7. 若干重复子层需要放在 `nn.ModuleList` 中，才能被 PyTorch 正确注册；
8. 模型参数量可以通过 `state_dict()` 或 `model.parameters()` 统计；
9. 模型和输入数据都必须位于同一个 device 上；
10. 在模型实现中，用 `assert` 检查 shape 是非常值得保留的工程习惯。

8 Lecture 3, Part I: Introduction, Starting Point, and How to Think About LM Architectures

8.1 Overview of This Part

这一部分对应 lecture 3 课件开头的内容，主要包括：

- 课程开场与 logistics;
- 本讲的目标 (outline and goals);
- 以 “original transformer” 作为起点进行快速回顾;
- 对比大家在作业中实现的 “simple, modern variant”;
- 引出本讲的核心问题：面对大量现代大语言模型，应该怎样看待架构设计？

这一部分在整份 lecture 中的作用，不是直接展开某个具体技术点，而是建立一个总框架：

1. 我们为什么需要这一讲；
2. 这一讲不是从零讲 Transformer，而是从你已经实现过的版本继续向前；
3. 现代大模型虽然种类很多，但不是每个细节都值得同等关注；
4. 最值得学习的是：哪些设计已经形成共识，哪些地方仍然在变化。

因此，这一部分的真正主题可以概括为：

从 “标准 Transformer” 出发，建立理解现代大语言模型架构选择的观察框架。

8.2 Opening and Logistics

lecture 开头先有标题页：

Lecture 3

Everything you didn't want to know about LM architecture and training

然后是一个很短的 logistics 页面，主要包括：

- Join the slack!
- Check to make sure you have the latest version of the assignment!

理解与注意事项 这一页技术内容很少，但它透露了 lecture 的一个背景：这一讲是和课程作业、课程讨论环境紧密关联的。也就是说，这不是一堂脱离实践的纯理论课，而是默认你已经：

- 在课程里实现过某个 Transformer 版本；
- 正在做相关 assignment；
- 接下来会把 lecture 中讲的设计选择和自己的实现联系起来。

这点非常重要，因为它决定了 lecture 3 的语气：它不是从头定义所有组件，而是在 “你已经做过一个版本” 的基础上，开始比较现代 LLM 的实际设计经验。

8.3 Outline and Goals

课件随后给出本讲目标：

- Quick recap of the “standard” transformer (what you implement)
- What do most of the large LMs have in common?
- What are common variations to the architecture / training process?

并给出一句非常重要的主题句：

Today’ s theme: the best way to learn is hands-on experience; the second best way is to try to learn from others’ experience.

理解与注意事项 这句话几乎可以看作这整讲的教学哲学。

第一层意思是：

最好的学习方式仍然是自己实现、自己训练、自己踩坑。

这和 CS336 整门课的风格完全一致——理解来自构建。

第二层意思是：

如果无法亲自复现所有大型模型，就要尽可能系统地学习他人的经验。

这恰好解释了为什么这一讲会大量讨论：

- 不同模型的共性；
- 常见的架构变体；
- 超参数选择；
- 稳定性技巧。

因为在 frontier-scale 模型无法被完整复现的前提下，“从公开模型与论文中抽取经验”本身就是一种很现实的学习方式。

这一页真正想建立的预期 这一页还在暗示一个很重要的学习姿态：

lecture 3 不是要你死记“哪个模型用了什么”，而是要你学会如何从大量模型设计中抽取规律。

因此，后面听这讲时，最重要的不是背表，而是不断问：

- 什么是大家都在做的？
- 什么是还没有共识的？
- 为什么会这样？
- 这和效率、稳定性、规模有什么关系？

8.4 Starting Point: The “Original” Transformer

lecture 接着明确指出起点:

Starting point: the “original” transformer

并快速回顾标准 Transformer 中的几个关键选择:

- Position embedding: sines and cosines
- FFN: ReLU
- Norm type: post-norm, LayerNorm

理解 这里其实是在提醒你: 如果回到最经典的《Attention Is All You Need》, 那么一个“原始 Transformer”的默认设计大致是:

- 位置编码: 正弦余弦位置编码;
- 前馈层激活: ReLU;
- 归一化方式: post-norm, 且使用 LayerNorm。

也就是说, 在 lecture 3 里, “标准 Transformer”不是一个模糊概念, 而是有明确参考系的:

sinusoidal position embedding + ReLU FFN + post-norm LayerNorm.

为什么这一页重要 这一页的作用不是复习教科书内容本身, 而是建立一个对照基线。因为后面 lecture 里讨论的所有现代变体, 几乎都可以表述成:

“与原始 *Transformer* 相比, 这里改成了什么?”

如果没有这个基线, 后面的 “modern LLM architecture” 就会变成一堆零散 tweak, 缺少清晰的参考坐标。

8.5 What You Implemented: A Simple, Modern Variant

lecture 紧接着做了第二个对照: 不是直接从 paper 里的 original Transformer 出发, 而是从你们在作业中实现的版本出发。

课件中列出的差异包括:

- LayerNorm is in front of the block
- Rotary position embeddings (RoPE)
- FF layers use SwiGLU, not ReLU
- Linear layers (and layernorm) have no bias (constant) terms

并问出两个问题:

- Why did we pick these?
- What should you pick?

理解 这意味着：你们作业里实现的并不是“最原始的 Transformer”，而是一个简化但更现代的版本。

也就是说，课程默认你已经接触过的 baseline，其实已经带有现代 LLM 的若干共性设计：

1. **pre-norm**: LayerNorm 放在 block 前面，而不是放在残差后面；
2. **RoPE**: 使用旋转位置编码，而不是正弦绝对位置编码；
3. **SwiGLU**: 前馈层激活不再是 ReLU，而是 gated activation 家族的一员；
4. **no bias**: 线性层与 LayerNorm 去掉 bias 项。

为什么课程先给这个版本 这背后有一个很重要的教学判断：

如果直接让学生实现最原始的 Transformer，虽然历史上更“正宗”，但离现代大模型实践反而更远。

因此，课程选择了一个“simple, modern variant”作为实现起点，使得学生：

- 仍然能掌握基本结构；
- 同时更接近现代大模型的主流设计。

注意细节 这里还要注意 lecture 的提问方式并不是：

“这些就是正确答案。”

而是：

“Why did we pick these? What should you pick?”

这表明 lecture 的态度很清楚：

- 这些设计不是无条件真理；
- 它们是当前经验上比较合理的选择；
- 你仍然应该理解其背后的原因，而不是机械照抄。

8.6 How Should We Think About Architectures?

课件接下来提出一个过渡性问题：

How should we think about architectures?

然后强调：

- Lots of architecture. Just in the last year since last 336..
- Over 19 new dense model releases, many of them with minor architecture tweaks..

理解 这一页的意思是：现代 LLM 世界里，模型非常多，名字非常多，改动也非常多。但如果只是把这些模型当成一堆独立产品来看，很容易陷入两个问题：

- 记不住；
- 即使记住了，也不知道哪些差异重要，哪些只是小修小补。

所以 lecture 在这里停下来先问一个更本质的问题：

面对大量架构变体，我们应该怎样建立自己的理解方式？

这一页真正想传达的学习方法 不是去追每一个新模型的 marketing，而是去做结构化观察：

- 哪些设计几乎所有模型都一样？
- 哪些设计仍然存在多种路线？
- 哪些改动只是 minor tweak？
- 哪些改动对训练稳定性、推理效率或规模扩展影响更大？

这就是后面整讲的主线。

8.7 Learn from the Data: Looking Across Many Models

lecture 随后明确说：

Let's look at the data (on dense architectures)

并进一步说明：

Learn from the many other models (and papers) out there

然后列出三个问题：

- What do all these models have in common?
- What parts vary?
- What can we learn from this?

理解 这里的“data”不是训练语料，而是：

模型设计本身构成的经验数据。

换句话说，lecture 的方法不是只读一篇理论 paper，而是把大量已发布 dense LLM 的设计汇总起来，像观察数据集一样观察它们。

这是一种非常重要的研究方法论：

当某个问题没有一个完全统一的理论答案时，可以先从经验分布中提炼规律。

例如你不一定先能证明“为什么所有现代模型都偏向某种 norm”，但你可以先观察：

- 大家到底在用什么；
- 哪些设计已经明显形成共识；
- 哪些地方仍然没有稳定答案。

注意细节 这里 lecture 说的是 “dense architectures”，说明它当前聚焦的是 dense LM，而不是 MoE 等更复杂路线。这意味着整讲的很多结论都应理解为：

“在现代 *dense LLM* 的设计经验中，普遍看到的模式。”

而不是对所有可能模型族都无条件成立。

8.8 What Are We Going to Cover?

课件随后给出本讲覆盖范围：

- Common architecture variations
 - Activations, FFN
 - Attention variants
 - Position embeddings
- Hyperparameters that (do or don't) matter
 - What is `ff_dim`? Do `multi_head` dims always sum to `model_dim`?
 - How many vocab elements?
- Stability tricks

理解 这一页非常重要，因为它告诉你 lecture 3 的范围并不是“讲完所有大模型知识”，而是集中在三类问题：

1. **架构变体**：block 里真正会改的东西；
2. **超参数**：维度、词表等设计是否有经验共识；
3. **稳定性技巧**：让训练不炸的工程方法。

这三类问题几乎就覆盖了现代 LLM 设计中最现实的决策层面。

这页背后的课程选择 值得注意的是，lecture 并没有把重点放在：

- multimodality；
- RAG；
- agent 框架；

- 产品层功能。

而是依然聚焦在：

语言模型本体的 **architecture and training**。

这和 CS336 整门课一贯的底层风格是一致的。

8.9 High-Level Preview of Architecture Variations

在正式进入后续内容前，课件先给了一个总览页：

Architecture variations.. Let's think about the core architecture piece

并给出高层判断：

- Low consensus (except pre-norm)
- Trends toward “LLaMA-like” architectures

理解 这句话非常值得仔细体会。

第一层意思是：

在很多架构细节上，其实没有一个所有人都同意的唯一答案。

也就是说，现代大模型虽然越来越趋同，但并不是每个细节都已经“定于一尊”。

第二层意思是：

尽管没有完全统一答案，整体趋势却在向某种 LLaMA 风格的配置收敛。

这为后面的细节讨论设定了语境：

- 后面要讲的很多变体，并不是完全平行的随机选择；
- 它们中有一些已经逐渐成为现代大模型默认配置；
- lecture 会在后面不断指出哪些地方共识更强，哪些地方仍然开放。

为什么这里先强调 “except pre-norm” 因为 lecture 已经提前暗示：在所有设计点里，**pre-norm** 是共识最强的那个之一。这等于给后面真正进入 architecture variations 做了铺垫。

8.10 What This Part Really Wants Us to Learn

从学习角度看，这一部分真正想传达的不是某个局部公式，而是以下几个更高层的认识。

8.10.1 (1) Use the original Transformer as a reference point

现代大模型的大部分设计讨论，都是相对于 “original Transformer” 来说的。因此必须先知道：

- 原始位置编码是什么；
- 原始 FFN 激活是什么；
- 原始 norm 放在哪里。

8.10.2 (2) Your assignment implementation is already a modernized baseline

课程作业里的版本并不是教科书最原始版本，而是一个简单但现代的 baseline。这意味着你后续学习现代 LLM 时，并不是从完全陌生的东西开始，而是在一个已经比较接近实际系统的实现上继续前进。

8.10.3 (3) Architecture comparison needs a framework

面对大量模型发布，最重要的不是“记住每家用了什么”，而是学会按以下问题整理：

- 什么是共识？
- 什么在变化？
- 哪些变化真的重要？

8.10.4 (4) This lecture is about extracting structured experience

lecture 3 的核心不是从零推导整个大模型理论，而是从许多已存在模型中抽取经验规律。这是一种很现实、也很有价值的学习方式。

8.11 Summary Notes

这一部分可以总结为以下学习笔记要点：

1. lecture 3 的目标是：从标准 Transformer 出发，系统讨论现代大语言模型在架构与训练上的共性和变体；
2. 本讲的学习方法论是：最好的学习是亲手实现，第二好的学习是系统吸收他人的模型设计经验；
3. original Transformer 可以被看作一个清晰基线，其典型特征包括：
 - 正弦余弦位置编码；
 - ReLU 前馈层；
 - post-norm + LayerNorm。
4. 课程作业中实现的版本已经带有明显的现代化设计，包括：
 - pre-norm；
 - RoPE；
 - SwiGLU；
 - 去 bias。
5. 面对大量现代 LLM，不应机械记忆每个模型，而应关注：
 - 哪些设计形成共识；

- 哪些地方仍然在变化；
- 能从这些经验中学到什么。

6. lecture 3 后续的主要范围包括：

- 架构变体；
- 重要与不重要的超参数；
- 稳定性技巧。

7. 从整体趋势上看，现代 dense LLM 在很多地方正向 “LLaMA-like” 配置收敛，但除少数点（如 pre-norm）外，许多架构细节仍然不存在绝对统一答案。

9 Lecture 3, Part II: Normalization Choices in Modern LLMs

9.1 Overview of This Part

这一部分对应 lecture 3 课件中关于 normalization 的整块内容，主要包括：

- **Pre-vs-post norm**
- 相关经验图与解释
- “double” norm / residual 外再加 norm 的新做法
- **LayerNorm vs RMSNorm**
- 为什么 RMSNorm 可能更快
- 更一般地去掉 bias terms
- 本部分 recap

这一部分在整份 lecture 3 中具有非常重要的地位。因为在所有架构选择中，课件一开始就明确给出一个高层判断：

High level view:

Low consensus (except pre-norm)

Trends toward “LLaMA-like” architectures

也就是说，现代大语言模型在许多架构细节上并没有完全统一答案，但 **normalization** 的若干选择已经形成了相当强的趋势，尤其是：

- 几乎都使用 **pre-norm**;
- 很多现代模型使用 **RMSNorm**;
- 更广义地，越来越多模型去掉 bias terms。

因此，这一部分的主线可以概括为：

现代 LLM 在 **normalization** 相关设计上，已经出现了比其他架构点更强的工程共识，而这些选择主要围绕训练稳定性与运行效率展开。

9.2 Pre-vs-Post Norm

9.2.1 The one thing everyone agrees on

课件首先给出一页标题：

Pre-vs-post norm

The one thing everyone agrees on (in 2024)

并写道:

Set up LayerNorm so that it doesn't affect the main residual signal path (on the left)

同时又明确指出:

Almost all modern LMs use pre-norm (but BERT was post-norm)

还补充了一个例外:

One somewhat funny exception - OPT350M. I don't know why this is post-norm

lecture 严格内容的含义 这里的核心信息非常明确:

1. 原始/早期 Transformer 体系里, post-norm 曾经是常见做法;
2. 但现代 LLM 几乎都切换到了 pre-norm;
3. 在课件给出的所有架构趋势中, pre-norm 是共识最强的一个。

什么是 pre-norm 与 post-norm 从结构角度看, 二者差异在于 LayerNorm 放在 residual block 的哪个位置。

Post-norm 结构中, 更接近原始 Transformer 的写法是:

$$x_{l+1} = \text{LN}(x_l + F(x_l)).$$

Pre-norm 结构中, 则更像:

$$x_{l+1} = x_l + F(\text{LN}(x_l)).$$

它们看起来只是 LayerNorm 位置不同, 但从优化动力学上看差异很大。

理解与注意事项 课件里那句

Set up LayerNorm so that it doesn't affect the main residual signal path

其实非常关键。

它想表达的是:

在 **pre-norm** 里, 残差主通路更像“干净直通”, LayerNorm 被放到了支路里, 而不是放在主残差和输出之间。

这对深层网络的梯度传播尤其重要, 因为 residual connection 的核心价值之一, 就是让信息和梯度能相对平滑地穿过很多层。如果把 norm 放在残差主通路之后, 就会在每一层都对主信号做一次额外变换, 这可能破坏残差结构的“直通性”。

9.2.2 The data pages

课件接着有一页标题：

Pre-vs-post-norm, the data

并引用了：

- Salazar and Ngyuen 2019
- Figure from Xiong 2020

理解 这一页的作用不是再给出新结论，而是说明：

pre-norm 的流行并不是拍脑袋决定，而是有经验数据支撑的。

也就是说，这个结论来自：

- 训练稳定性观察；
- 梯度行为对比；
- 大模型实践经验；
- 相关论文中的实证。

作为学习笔记，这里要忠实保留一点：lecture 并没有在这一页中自己推导新数学，而是通过已有论文图表来说明：

pre-norm 的优势是有经验支持的。

9.2.3 Explanations: why pre-norm?

随后课件有一页：

Pre-vs-post norm, explanations?

并列：

- Gradient attenuation [Xiong 2020]
- Gradient spikes [Salazar and Ngyuen]
- Original stated advantage - removing warmup.
- Today - stability and larger LRs for large networks

严格记录 lecture 内容 这一页给出了几类解释框架：

1. **Gradient attenuation:** 后归一化可能导致梯度传播时衰减；
2. **Gradient spikes:** 也可能导致梯度尖峰或不稳定；
3. 历史上早期讨论过的一个优点是：减少或去除 warmup 的需求；
4. 但在今天，lecture 更强调的是：大网络训练时的稳定性与支持更大学习率。

理解与注意事项 这一页最重要的是最后一句：

Today - stability and larger LR for large networks

这说明 lecture 的态度不是停留在历史解释，而是从现代大模型训练角度重新评价 pre-norm。

也就是说，在今天的 LLM 实践里，pre-norm 更重要的价值是：

- 深层网络更稳；
- 训练更不容易炸；
- 可以承受更大的 learning rate；
- 对大规模训练更友好。

一个很重要的理解 这里不要把“pre-norm 更稳定”简化成某种神秘经验。更准确地说，它反映了这样一个设计原则：

在极深网络中，要尽量保护 residual 主路径的可传播性。

pre-norm 正是顺着这个原则得到的一种结构安排。

9.3 New Things: “Double” Norm

课件随后给出一个新页：

New things - “double” norm.

If putting LayerNorms in residual streams is bad.. Why not post-norm outside the stream?

并提到：

Recent models: Grok, Gemma 2. Olmo 2 only does non-residual post norm

lecture 内容的严格含义 这部分想表达的是：如果大家已经接受“不要在 residual 主通路里做 post-norm”，那么是否可以考虑一种折中设计：

- block 内部仍然保持 pre-norm；
- 但在 residual 结构之外、或者某个非残差位置，再额外加一次 norm。

也就是说，这不是回到经典 post-norm，而更像：

保留 pre-norm 的主结构，同时在 block 外额外做一次归一化。

理解与注意事项 这里要特别注意 lecture 的措辞。它并没有说现代模型重新回到了 post-norm, 而是在说:

有些新模型尝试在 residual 主通路之外增加额外 norm。

这 and 传统 post-norm 不是一回事。

因此做笔记时要避免误写成:

“现代模型又开始用 *post-norm* 了。”

更准确的说法应该是:

“在 *pre-norm* 已成共识的背景下, 一些模型尝试额外加一层不位于 *residual* 主路径上的后置归一化。”

这也是 lecture 特别写出:

Olmo 2 only does non-residual post norm

的原因。

9.4 LayerNorm vs RMSNorm

课件接着进入下一组核心对比:

LayerNorm vs RMSNorm

并写道:

Original transformer: LayerNorm - normalizes the mean and variance across d_{model}

同时列出 notable models:

- GPT3/2/1, OPT, GPT-J, BLOOM

然后又说:

Many modern LMs: RMSNorm - does not subtract mean or add a bias term

并列 out notable models:

- LLaMA-family, PaLM, Chinchilla, T5

课件还给出 RMSNorm 的形式:

$$y = \frac{x}{\sqrt{\mathbb{E}[x^2] + \epsilon}} \cdot \gamma$$

从版面上看, 它强调的核心是:

- 只基于均方根尺度进行归一化;
- 不做均值中心化;
- 不加 bias。

理解 LayerNorm 标准 LayerNorm 通常会：

1. 先减去均值；
2. 再除以标准差；
3. 再做可学习缩放（以及常见版本中的 bias）。

因此它会同时处理：

- shift（均值）
- scale（方差）

理解 RMSNorm 而 RMSNorm 只做 scale 层面的归一化，不显式做均值中心化。换句话说：

RMSNorm 更像是在控制向量整体尺度，而不是把它强行挪到零均值。

lecture 在这里真正强调什么 不是说 RMSNorm 在数学上更“高级”，而是说：

很多现代大模型发现，不做完整 **LayerNorm**，而只做 **RMSNorm**，实践中也够用了。

这正是工程系统里常见的趋势：如果一个更简单、更便宜的模块效果差不多，那么它往往会逐渐替代更重的版本。

9.5 Why RMSNorm?

课件接着专门讨论：

Why RMSNorm?

并给出一句总结：

Modern explanation - it's faster (and just as good).

接着列出两点：

- Fewer operations (no mean calculation)
- Fewer parameters (no bias term to store)

然后又追问：

Does this explanation make sense?

理解 这一页很有意思，因为 lecture 并不是机械接受 “RMSNorm 更快” 这个说法，而是在提醒你继续思考：

既然 **matmul** 才是主导 **FLOPs**，大模型里少一点 **mean** 计算，真的会显著影响 **runtime** 吗？

随后课件自己也写：

Matrix multiplies are the vast majority of FLOPs (and memory)

并引用 Ivanov et al. 2023。

注意细节 这一页最重要的不是 “RMSNorm 快” 这句口号，而是 lecture 提出的怀疑：

如果只用 **FLOPs** 来思考问题，这个解释似乎不够完整。

这正为下一页做铺垫。

9.6 Why RMSNorm (2): FLOPs are not runtime

下一页课件标题是：

Why RMSNorm (2)

并明确写出一句非常重要的话：

Important lesson: FLOPS are not runtime! (we will discuss this in far more detail later)

然后说：

RMSNorm can still matter due to the importance of data movement

严格记录 lecture 的意思 这一页给出的核心修正是：

一个操作是否影响实际运行时间，不只取决于它贡献了多少 **FLOPs**，还取决于它的数据搬运成本。

也就是说，尽管 LayerNorm / RMSNorm 相对 **matmul** 来说 **FLOPs** 很少，但 **norm** 仍可能在 **wall-clock time** 上有可见差异，因为：

- 需要读写大量激活；
- **memory bandwidth** 可能成为瓶颈；
- 少一个统计量、少一点参数、少一点访存，都可能产生真实收益。

这是 lecture 中非常重要的一个 **general lesson** 这其实已经超越 RMSNorm 本身，而是在教你一个更广义的系统观：

不能只用 **FLOPs** 思考性能，必须把 **data movement** 一起纳入考虑。

这和 lecture 02 中讲 “FLOPs are not runtime” 的思想是高度一致的。

9.7 RMSNorm Validation

课件接着给一页：

RMSNorm - validation

并写：

RMSNorm runtime (and surprisingly, perf) gains have been seen in papers

并引用 Narang et al. 2020。

理解 这一页的意思是：

- 前一页是原理层面的解释：为什么 RMSNorm 可能更快；
- 这一页是经验层面的验证：论文中确实观察到 runtime 收益，甚至有时还有性能收益。

注意细节 lecture 特别用了 “surprisingly” 这个词，说明一件事：

RMSNorm 不只是 “更便宜但效果差不多”，在某些实验里甚至还可能更好。

当然，作为笔记要忠实记录：lecture 并没有把它绝对化成 “RMSNorm 总是更优”，而是在说：

“已有论文确实观察到了 *runtime gain*，且有时性能也不差甚至更好。”

9.8 More Generally: Dropping Bias Terms

课件随后把话题从 RMSNorm 推广到更一般的趋势：

More generally: dropping bias terms

并明确写道：

Most modern transformers don't have bias terms.

同时对比原始 Transformer 与更常见的现代实现，并指出：

Reasons: memory (similar to RMSnorm) and optimization stability

严格记录 lecture 内容 这一页想表达的是：

1. 现代模型中，不只是 RMSNorm 不要 bias；
2. 更广泛地，很多线性层和相关模块也倾向于去掉 bias；
3. 其原因主要包括：
 - 节省内存 / 减少参数搬运；
 - 以及某种优化稳定性方面的考虑。

理解与注意事项 这里不能把“去 bias”理解成节省参数量特别巨大。从绝对参数量上看，bias 往往比大矩阵小得多。lecture 真正想强调的是：

在现代 LLM 中，很多设计选择都在朝“去掉那些收益不大但仍有访存/稳定性代价的附加项”这个方向走。

换句话说，去 bias 和 RMSNorm 的逻辑是一致的：

- 不是因为 bias 本身 FLOPs 特别大；
- 而是因为在超大模型里，任何多余参数和多余访存最终都会累积成真实成本；
- 如果它们的收益不明显，就倾向于删掉。

9.9 LayerNorm Recap

这一部分最后课件有一个 recap 页面，内容非常重要，因为它明确给出本部分结论。课件写道：

- Basically everyone does pre-norm.
 - Intuition - keep the good parts of residual connections
 - Observations - nicer gradient propagation, fewer spike
 - Some people add a second norm outside the residual stream (NOT post-norm)
- Most people do RMSnorm
 - In practice, works as well as LayerNorm
 - But, has fewer parameters to move around, which saves on wallclock time
 - People more generally drop bias terms since the compute/param tradeoffs are not great.

这页的意义 这页几乎可以看作本部分的标准答案。

它把 normalization 相关选择压缩成两条最重要的经验结论：

1. **pre-norm** 几乎是现代 LLM 的默认共识
2. **RMSNorm** 和去 **bias** 是现代工程实践中非常常见的选择

理解与注意事项 尤其要注意 recap 里这几句：

keep the good parts of residual connections

和

works as well as LayerNorm

以及

fewer parameters to move around, which saves on wallclock time

这三句分别对应本部分三条最核心的逻辑:

- **pre-norm** 的核心是优化与梯度传播逻辑;
- **RMSNorm** 的核心是 “效果不差”;
- **RMSNorm** / 去 bias 的工程价值在于更少参数搬运与更好 wall-clock efficiency。

9.10 What This Part Really Wants Us to Learn

从学习角度看, 这一部分真正想传达的是以下几点。

9.10.1 (1) Pre-norm is one of the strongest modern LLM consensuses

在大量架构点上没有完全统一答案, 但 pre-norm 基本已经成为现代大模型的默认选择。理解这一点, 比记住某个单独模型用了什么 norm 更重要。

9.10.2 (2) Residual-path thinking is central

lecture 对 pre-norm 的解释本质上是在强调:

设计 norm 时, 要优先保护 residual 主通路的可传播性。

这是理解 pre-norm 背后优化优势的关键。

9.10.3 (3) RMSNorm is an engineering simplification that often works

RMSNorm 不是为了炫技, 而是一个非常典型的工程决策:

- 更简单;
- 参数更少;
- 访存更少;
- 效果通常不差。

9.10.4 (4) FLOPs do not fully explain runtime

这一部分最重要的系统课之一是:

FLOPs are not runtime.

如果只看 FLOPs, 很难解释为什么 RMSNorm 会有实际速度收益; 但一旦把 data movement 考虑进来, 这个问题就清楚了。

9.10.5 (5) Dropping bias is part of a broader design trend

现代模型去掉 bias, 不是一个孤立小 tweak, 而是更大趋势的一部分:

删去收益不明显、但会增加参数搬运与复杂度的部件。

9.11 Summary Notes

这一部分可以总结为以下学习笔记要点：

1. 在现代 dense LLM 中，pre-norm 是最强的架构共识之一，而 post-norm 主要属于较早的 Transformer 设计；
2. pre-norm 的核心直觉是：让 LayerNorm 不直接干扰 residual 主通路，从而改善深层网络中的梯度传播与训练稳定性；
3. 课件中提到的 pre-norm 优势包括：
 - 减少 gradient attenuation；
 - 减少 gradient spikes；
 - 在现代大网络中支持更高学习率与更稳定训练。
4. 一些新模型在 pre-norm 基础上增加了 residual 之外的额外 norm，但这不等于回到了传统 post-norm；
5. LayerNorm 会处理均值和方差，而 RMSNorm 主要只控制 RMS 尺度，不显式减均值，也通常不带 bias；
6. 许多现代模型使用 RMSNorm，因为它实践中通常和 LayerNorm 表现相近，同时参数更少、访存更少；
7. lecture 特别强调：**FLOPs are not runtime**，因此 norm 层的 wall-clock 差异必须结合 data movement 一起理解；
8. 现代 Transformer 中广泛去掉 bias terms，其理由与 RMSNorm 类似，主要和参数搬运成本及优化稳定性有关；
9. 本部分的整体结论是：现代 LLM 在 normalization 相关设计上，已经明显朝着 “pre-norm + RMSNorm + no bias” 这一类 LLaMA-like 配置收敛。